

Re-architecture of the DisloCluster 0-D Solver and of the 0-D $\leftrightarrow$ 3-D Coupling

DisloCluster development note

8 August 2026

Abstract

Three changes were made to the DisloCluster 0-D side and to its coupling with the spatially resolved code. (i) The right-hand side was moved into a single scalar-type-templated core so that the residual and an *exact* Jacobian, obtained by forward-mode automatic differentiation, are two instantiations of one definition; SUNDIALS' difference-quotient Jacobian is thereby retired. (ii) The six point-defect conservation accumulators were recognized as pure quadratures and moved out of the implicit system into CVODES quadrature variables, shrinking the Newton block from 19×19 to 13×13 standalone and to 9×9 under the operator split. (iii) The 0-D $\rightarrow$ 3-D handoff, which previously collapsed the marched state to four scalars, was replaced by a genuine per-node field exchange through MoDELib3's own cluster-dynamics configuration block.

Measurement then redirected the effort: at $n = 19$ the dense linear algebra was never the bottleneck. Profiling with the integration window collapsed to zero showed that per-case construction and destruction of SUNDIALS objects consumed 1.31 s of thread time against 0.2 s of actual solving in a 256-point coupling substep. Reusing one solver workspace per OpenMP thread cut the end-to-end march wall time by a factor of 1.67 at $q = 1024$ quadrature points — more than the two originally requested changes combined. The report states all three outcomes as measured, including the two that did not pay off in wall time.

Two further sections document the coupling in operation. Section 6.6 measures the full 1–26 dpa march on the 1 μm single crystal in three configurations, separating the genuine solver speedup ($1.95\times$ at identical work) from the cost of resolving a field on 24115 nodes instead of averaging 25 line points to four scalars. Section 7 characterizes MoDELib3's own nodal immobile integrator against the CVODE march — correcting, along the way, the common description of it as explicit Euler: it is a first-order semi-implicit scheme, and it has to be, because the measured stiffness puts its step 46 000 times beyond the forward-Euler limit. Section 8 then traces where each scheme came from, against the correct upstream: `mlm335/MoDELib-fullCD`, which already carries a full immobile field and its own IMEX integrator. MoDELib3's immobile solver is therefore a re-discretization of an existing scheme — nodal collocation and a single fused semi-implicit update, in place of fullCD's Galerkin mass-matrix projection followed by explicit Euler and sequential implicit loss factors — and not, as an earlier revision of this report stated on the strength of the wrong repository, an invention from nothing.

A later revision corrects the coupled route itself. As first written it did not perform an operator split: it read its mobile field out of a previously recorded standalone run, so $\mathbf{C}_M$ never responded to the immobile state the march was building. Section 10.1 puts MoDELib3's own steady mobile solve back inside the loop, through a new `useImmobileSolver` switch that lets a DDomp run perform the fast step alone. Section 10.3 then measures what that repairs: a 0-D seed at 0.1 dpa carries a C_v some $690\times$ above the quasi-steady value and no boundary layer at all, and the fast step supplies both corrections on its first call.

Contents

1 Starting point

3

2	Structural observation: the accumulators are pure quadratures	4
3	Implementation	4
3.1	One templated core, two instantiations	4
3.2	Reduced implicit block	5
3.3	Reusable per-thread solver workspace	5
3.4	Interface	5
4	Verification	5
4.1	The AD Jacobian	5
4.2	Bit-identity of the untouched path	6
5	Results	6
5.1	Standalone 0-D run	6
5.2	Operator-split coupling march	7
5.3	Where the improvement actually came from	7
5.4	A step-size pathology the reduction cures	8
5.5	The quadrature memo, and why the reduction does not win on time	8
6	The 0-D $\leftrightarrow$ 3-D field handoff	9
6.1	What was wrong	9
6.2	What MoDELib3 actually offers	9
6.3	The mapping	9
6.4	Verification on the 3-D verification case	10
6.5	Notebook wiring	10
6.6	Measured cost of the field upgrade	10
7	Integrator comparison: MoDELib’s nodal update versus the CVODE march	11
7.1	What MoDELib actually does	12
7.2	Test method	12
7.3	Stability	12
7.4	Accuracy and speed	13
7.5	Consequence for the coupling	14
8	Provenance: how the two schemes came to be	14
8.1	The upstream scheme, and how MoDELib3 re-discretized it	15
8.2	How MoDELib3’s variant reached its present form	16
8.3	The coupled march scheme: outward from the 0-D solver	16
8.4	Three routes to the same conclusion	17
9	The three recommendations, implemented and measured	18
9.1	Recommendation 1: how the accumulators are carried	18
9.2	Recommendation 2: the base+delta case protocol	19
9.3	Recommendation 3: deduplicating points before dispatch	19
9.4	Combined effect	20
10	Two options for time integration by operator splitting	20
10.1	The fast step belongs inside the loop	21
10.2	A non-terminating mobile solve, and its cause	21
10.3	Does the split repair a seed that is not quasi-steady?	23
10.4	A calibration error that voids the earlier physics numbers	23
10.5	The whole-domain mean is the wrong statistic	24
10.6	Does the seed dose matter? Seeding inside and outside the transient	26

10.7 Removing the 0-D seed entirely	28
10.8 What this does and does not isolate	29
10.9 The two routes, compared	30
10.10 Speed	30
10.11 Attribution: model, splitting, or integrator?	31
10.12 A discrepancy with the calibration record	31
10.13 Output	32
11 Why the slow step is tractable	32
11.1 The decoupling is a property of the split, not of the solver	32
11.2 What the decoupling is worth	32
11.3 Where the tight coupling went	33
11.4 The price, and the check	33
12 Construction of Discrete Loops	34
12.1 What the earlier figures were, and were not	34
12.2 Mathematical method	34
12.3 Coalescence, and why it must be bounded	35
12.4 Differences from MoDELib-fullCD	35
12.5 Interface with the dislocation dynamics input files	36
12.6 Results	38
12.7 From an offline export to a runtime handoff	38
13 A self-consistent slow solver	40
13.1 The inconsistency	40
13.2 One switch, two formulations	40
13.3 Where the parameters come from, and why it matters	41
13.4 Verification: bit-identity of the fitted path	41
13.5 What the formulation change costs, and the attribution	41
13.6 One combination is refused	42
14 Summary and recommended next steps	42

1 Starting point

The 0-D model integrates $N_{\text{eq}} = 19$ ordinary differential equations: 12 physical species, 6 point-defect conservation accumulators, and the evolving network dislocation density ρ_N . In the operator-split coupling the four mobile species $\mathbf{C}_M = [C_v, C_i, C_{2i}, C_{3i}]$ are frozen at the value supplied by the 3-D fast solve (`freeze_mobile=1`), and the remaining state is marched independently at every quadrature point, one case per point in an OpenMP batch subprocess.

Before this work the solver had the following properties.

- **No Jacobian was supplied.** `CVodeSetJacFn` was never called, so SUNDIALS formed the 19×19 dense Jacobian by difference quotients, costing 19 extra residual evaluations per Jacobian setup.
- **All 19 equations were implicit.** The accumulators were carried as ODE state, entered the Newton system and the error test, and were factorized in every LU.
- **Every case built its own solver.** `integrate_one` created and destroyed a `SUNContext`, state vector, dense matrix, linear solver and CVODE memory per invocation — i.e. once per quadrature point per substep.

- **The 0-D $\rightarrow$ 3-D carrier was four scalars.** The driver averaged the marched state over all points and wrote one number density and one mean radius per loop family into MoDELib’s microstructure generator inputs.

2 Structural observation: the accumulators are pure quadratures

Rows 12–17 of the right-hand side are written but columns 12–17 are never read: no equation in the system depends on an accumulator value. Inspection of `rate_equations.cpp` confirms that `y[12..17]` appears nowhere on the right of any assignment. The Jacobian therefore has the block structure

$$J = \begin{pmatrix} J_{ss} & 0 \\ J_{as} & 0 \end{pmatrix}, \quad s = \{\text{species}, \rho_N\}, \quad a = \{\text{accumulators}\}, \quad (1)$$

so the accumulators are quadratures of the state trajectory, $\mathbf{a}(t) = \int \mathbf{g}(\mathbf{y}_s(\tau)) d\tau$, and belong in CVODES’ quadrature module rather than in the Newton system.

Combining this with the operator split, which freezes the four mobile species, gives the implicit block dimensions of Table 1.

Table 1: Dimension of the implicit block actually factorized.

Configuration	Newton block n	Dense LU $\propto n^3$	Relative LU cost
Legacy (all 19 implicit)	19	6859	1.00
Reduced, standalone	13	2197	0.32
Reduced, <code>freeze_mobile=1</code>	9	729	0.11

3 Implementation

3.1 One templated core, two instantiations

The physics was moved verbatim from `rate_equations.cpp` into `rate_equations_core.h` as

```
template <class T> void rhs_core(const T* y, T* ydot, const Parameters& P)
```

instantiated with `T = double` for the residual and with `T = Dual<N>` (`dual.h`) for the Jacobian. Forward-mode automatic differentiation evaluates value and all N directional derivatives in a *single* sweep: `sqrt`, `exp` and `cbrt` are evaluated once and their derivatives propagated by a multiply, rather than the N complete residual evaluations a difference quotient needs.

Two properties motivated AD over a hand-written analytic Jacobian. First, exactness: there is no perturbation size to choose and no truncation error. Second, and more important for maintenance, the Jacobian cannot drift out of sync with the residual — hand-differentiating the coalescence Avrami gates and the smoothstep minimum-size gate would have created a second copy of the physics to keep aligned, on a code base that already maintains a Python mirror.

Bit-identity. Floating-point addition is not associative, so the transcription preserved the exact grouping of every expression. Where the original wrote `k1 * k2 * C * D`, meaning $((k_1 k_2)C)D$, the templated core hoists $k_1 k_2$ into a named `double` and keeps the remaining order. This was verified rather than asserted: a reference binary was built from the git-HEAD sources and the outputs compared byte for byte (Section 5).

3.2 Reduced implicit block

With `-reduced=1` the state vector holds only the implicit components and the accumulators are advanced by `CVodeQuadInit`. Quadrature error control is left at its default (off), so the accumulators ride the step sequence the state equations demand instead of constraining it. The 19-column output contract is unchanged: each row is reassembled from the reduced state, the frozen mobile values and the quadrature vector.

3.3 Reusable per-thread solver workspace

This change was not planned; it was the result of profiling. Running a 256-case frozen-mobile batch with the integration window collapsed to $t_{\text{end}} = t_{\text{begin}}(1 + 10^{-12})$ — so that only process spawn, case-file parsing and output formatting remain — gave the decomposition in Table 2.

Table 2: Cost decomposition of one 256-case coupling substep, before the workspace change. “cpu-solve” is the sum of per-case integration wall times over all threads.

Configuration	wall [s]	cpu-solve [s]
Null integration (setup/teardown only)	0.172	1.310
1 dpa, 19×19 difference-quotient	0.136	1.608
1 dpa, 19×19 AD	0.129	1.491

Constructing and destroying the SUNDIALS objects cost 1.31 s of thread time against roughly 0.2 s of genuine integration: the per-case setup was consuming about six times what the ODE solve did. A `Workspace` is now created once per OpenMP thread and reused across every case that thread handles via `CVodeReInit` / `CVodeQuadReInit`, rebuilt only when a case needs a different shape. In a 1024-point march this reduces 1024 solver constructions per substep to 24, one per thread.

3.4 Interface

Three flags were added; both numerical flags default to *off* so that a standalone 0-D run remains bit-identical to the previous solver. `modelib_coupling.build_immobile_cases` turns them on for the march, where they are measurably better.

Flag	Effect
<code>-reduced=1</code>	Accumulators as CVODES quadrature; Newton block 13 (or 9 with <code>freeze_mobile</code>). CVODE only.
<code>-analytic_jac=1</code>	Exact dense Jacobian by forward AD. Dense linear solver only.
<code>-stats=1</code>	Emit a <code># STATS</code> comment line with the integrator counters. Non-numeric, so the existing Python row parsers skip it.

4 Verification

4.1 The AD Jacobian

A standalone tool (`jac_check.cpp`) compares the AD Jacobian against a central-difference Jacobian of the *same* core, differencing column j with a relative step $h_j = \varepsilon_{\text{rel}}|y_j|$ because the state spans some 34 decades. It was run at 20 states sampled along a converged trajectory from 10^{-8} to 100 dpa.

Two classes of entry are excluded, for reasons belonging to the finite difference and not to the AD:

- **FD-unresolvable entries.** A central difference can only see an entry whose contribution $J_{ij}h_j$ rises above the floating-point granularity of f_i . At the initial condition, dC_i/dt is pinned near $G_i \approx 10^{-7}$ while its C_v derivative moves it by $\sim 10^{-28}$, i.e. 10^{-5} of one unit in the last place. The difference quotient returns exactly zero there at *any* step size; AD returns the exact value. Entries are compared only where the signal exceeds 100 ULP. Note that this is precisely the blindness SUNDIALS’ difference-quotient Jacobian suffers, so the AD Jacobian is strictly more informative in this region.
- **Components on the concentration floor**, where `f1()` makes the residual non-smooth: AD returns the one-sided derivative while a central difference straddles the kink.

The result is in Table 3. The discrepancy falls by two decades per decade of ε_{rel} across the usable range, the $O(h^2)$ signature of a correct derivative with the finite difference’s own truncation error dominating.

Table 3: AD versus central-difference Jacobian, worst row-relative discrepancy over all compared entries. Representative states from the 20 tested.

Dose [dpa]	$\varepsilon_{\text{rel}} = 10^{-5}$	10^{-6}	10^{-7}	entries compared
1.0×10^{-8}	6.50×10^{-12}	8.84×10^{-11}	1.24×10^{-9}	6
1.3×10^{-6}	8.49×10^{-8}	7.96×10^{-7}	2.05×10^{-6}	108
1.6×10^{-4}	8.29×10^{-8}	9.97×10^{-7}	1.83×10^{-5}	117
2.1×10^{-2}	4.40×10^{-7}	4.71×10^{-6}	5.16×10^{-5}	133
7.8×10^{-1}	1.55×10^{-7}	9.30×10^{-7}	9.92×10^{-6}	133
8.9×10^0	3.09×10^{-9}	4.28×10^{-8}	1.30×10^{-6}	107
1.0×10^2	8.04×10^{-9}	5.17×10^{-8}	5.71×10^{-7}	107
Worst over all 20 states (best ε per state): 4.40×10^{-7} ; states failing at every ε : 0/20.				

4.2 Bit-identity of the untouched path

The legacy configuration of the rebuilt solver reproduces the git-HEAD reference binary *byte for byte* — 1000 output rows $\times$ 19 columns in the standalone run, and the endpoint state of all 256 points $\times$ 5 substeps in the coupling march. The refactor into a templated core therefore changed no arithmetic.

5 Results

All runs use `input/Zr_input_parameters.xlsx` at $T = 573$ K, $G = 10^{-7}$ dpa/s, on 24 cores. Because wall-clock times on a shared machine are noisy, exact operation counts are reported alongside and are the primary evidence:

$$\begin{aligned}
 n_{\text{core}} &= \text{residual-core evaluations} = \text{nfe} + \text{nfeLS} + \text{nqmiss}, \\
 n_{\text{LU}} &= \text{dense factorizations (nsetups)}, \quad \text{LU flops} \propto n_{\text{LU}} n^3/3.
 \end{aligned}$$

5.1 Standalone 0-D run

$t = 10^{-1} \dots 10^9$ s (100 dpa), 1000 output points, $\text{rtol} = 10^{-6}$, $\text{atol} = 10^{-20}$. Accuracy is measured against a tight reference ($\text{rtol} = 10^{-10}$, $\text{atol} = 10^{-24}$), compared per column only where the reference exceeds 10^{-8} of that column’s own peak — a plain elementwise relative error is meaningless on a state spanning 34 decades whose early transient sits below atol .

The AD Jacobian removes 32% of the residual evaluations ($6129 \rightarrow 4171$) and 11% of the steps. The reduction cuts LU work by 71% but *adds* residual evaluations, for the reason given

Table 4: Standalone 0-D run. “bit=ref” is byte-for-byte agreement with the git-HEAD binary.

Configuration	wall [ms]	solve [ms]	nst	n_{core}	n_{LU}	LU flops	bit=ref
git HEAD 19×19 DQ	70.7	–	–	–	–	–	–
Baseline 19×19 DQ	69.3	46.7	2797	6129	466	1.07×10^6	yes
AD Jac. 19×19 AD	65.5	46.9	2483	4171	429	9.81×10^5	no
Reduced 13×13 DQ	64.5	42.3	2564	7823	423	3.10×10^5	no
Reduced + AD 13×13	63.4	43.7	2811	7415	458	3.35×10^5	no

Table 5: Standalone accuracy against the tight reference (max relative error).

Configuration	species	accumulators	ρ_N
git HEAD / baseline 19×19 DQ	1.41×10^{-4}	2.15×10^{-5}	1.60×10^{-5}
AD Jac. 19×19	2.18×10^{-4}	4.90×10^{-5}	3.16×10^{-5}
Reduced 13×13 DQ	2.44×10^{-4}	5.25×10^{-5}	3.30×10^{-5}
Reduced + AD 13×13	8.79×10^{-5}	2.18×10^{-5}	1.30×10^{-5}

in Section 5.5. All four configurations sit within a factor of three of each other in accuracy, at the level expected from different step sequences at the same tolerance; none is meaningfully more accurate than another.

5.2 Operator-split coupling march

A 5 dpa dose step in 5 substeps, seeded at 1 dpa, with the mobile field depleted toward a sink across the point set. Table 6 gives $q = 1024$; the $q = 64$ and $q = 256$ rows scale proportionally.

Table 6: Coupling march, $q = 1024$ quadrature points, 5 substeps. “warn” counts CVODE roundoff warnings (see Section 5.4); error is against the git-HEAD result.

Configuration	wall [s]	cpu-solve [s]	nst	n_{core}	n_{LU}	LU flops	warn
git HEAD 19×19 DQ	2.379	–	–	–	–	–	2048
Reuse 19×19 DQ	1.424	12.03	175771	366170	69860	1.60×10^8	2048
Reuse + AD 19×19	1.435	12.98	175890	243682	69921	1.60×10^8	2048
Reduced 9×9 DQ	1.568	13.99	143651	400404	54959	1.34×10^7	0
Reduced + AD 9×9	1.572	13.68	143654	349626	54957	1.34×10^7	0

Agreement with the git-HEAD march is 1.3×10^{-5} or better in every species at every q — the expected splitting-consistent difference between step sequences, not a change of answer.

5.3 Where the improvement actually came from

Stated plainly, and against the expectation that motivated the work:

- **Workspace reuse is the whole wall-clock gain:** $1.67\times$ at $q = 1024$, and it grows with q because the per-case setup it removes scales with the point count while the per-thread construction does not.
- **The AD Jacobian delivers its target metric:** residual evaluations fall by 33% ($366\,170 \rightarrow 243\,682$) and difference-quotient evaluations go to zero. Wall time is neutral, because one Dual<19> sweep costs about what the 19 scalar evaluations it replaces cost. Its durable value

Table 7: March wall time versus point count. Speedup is against the git-HEAD binary at the same q .

Configuration	$q = 64$		$q = 256$		$q = 1024$	
	[s]	speedup	[s]	speedup	[s]	speedup
git HEAD	0.254	1.00	0.692	1.00	2.379	1.00
Reuse 19×19 DQ	0.214	1.19	0.482	1.44	1.424	1.67
Reuse + AD 19×19	0.211	1.20	0.468	1.48	1.435	1.66
Reduced + AD 9×9	0.217	1.17	0.502	1.38	1.572	1.51

is exactness, the elimination of the FD-blind entries of Table 3, and a Jacobian that cannot fall out of step with the residual.

- **The 9×9 reduction delivers its target metric — a 92% cut in LU flops — but LU was never the bottleneck at $n = 19$.** It also reduces the step count by 18%. It nonetheless loses slightly in wall time, because of the quadrature cost analyzed next.

5.4 A step-size pathology the reduction cures

The `warn` column of Table 6 is not cosmetic. The legacy path emits, on every substep of every point,

[WARNING] Internal $t = 9.88525\text{e}+06$ and $h = 2.10129\text{e}-12$ are such that $t + h = t$ on the
next step

2048 of them in a 1024-point, 5-substep march.

The cause is the accumulators. Each substep restarts at $t \approx 10^7$ s with the accumulators reset to zero, and with $\text{atol} = 10^{-20}$ their error weights $w_i = 1/(\text{rtol}|y_i| + \text{atol})$ are $\sim 10^{20}$. CVODE’s initial-step heuristic uses the weighted norm of $\dot{\mathbf{y}}$; with $\dot{a} \sim 10^{-7}$ this gives $\|\dot{\mathbf{y}}\|_{\text{WRMS}} \sim 10^{13}$ and hence $h_0 \sim 10^{-13}$, while $\varepsilon t \approx 2 \times 10^{-10}$. The first step is below the representable resolution of t and is wasted.

Removing the accumulators from the error test eliminates this outright: zero warnings, and 18% fewer internal steps. This is the reduction’s most defensible benefit — it is a robustness property, independent of problem size, whereas the LU saving is not.

A second, subtler effect works the other way. The error test is a root-mean square over components, $\|e\|_{\text{WRMS}} = \sqrt{N^{-1} \sum_i (e_i w_i)^2}$, so removing six well-behaved components makes the norm $\sqrt{19/9} = 1.45$ times more sensitive per remaining component. The 19-equation system was being flattered by six dummy components diluting its error norm; the reduced system is held to a genuinely tighter standard at the same nominal tolerance.

5.5 The quadrature memo, and why the reduction does not win on time

CVODES calls the quadrature right-hand side once per completed step. Because that call needs the same core evaluation the residual has just performed, the implementation memoizes the accumulator rates on the exact $(t, \mathbf{y})$ bit pattern and reuses them on a match.

The measured hit rate is poor: 1 hit against 2564 misses in the standalone run. CVODE’s corrector evaluates the residual at its Newton iterates, and the accepted step point $\mathbf{y}_n$ is generally not one of them, so the memo misses and a second full core sweep is paid per step. That is exactly the +28% in n_{core} visible in Table 4, and it cancels the LU saving.

The identified remedy, not implemented here, is to keep the accumulators in the state vector but neutralize their error weights with a vector `atol` (`CVodeSVtolerances` with $\text{atol} = 10^{300}$ on those six components). That would retain the step-size cure of Section 5.4 at *zero* extra core

evaluations, at the cost of a 15×15 rather than 9×9 block — which, on this evidence, is the better trade. It is recorded as the next step rather than claimed as a result.

6 The 0-D $\leftrightarrow$ 3-D field handoff

6.1 What was wrong

The previous 0-D $\rightarrow$ 3-D carrier (`modelib_fem.write_sink_field`) wrote four scalars per dose step — one number density and one mean radius per loop family — into `inputFiles/aLoopsDensity.txt` and `inputFiles/frankLoopsDensity.txt`. Those files drive MoDELib’s *microstructure generator*, which places discrete loops at a uniform target density. The driver therefore had to collapse the marched state to its volume average before handing it over:

```
A = np.mean(np.asarray(Y), axis=0)
```

Every bit of spatial structure the slow march produced was discarded on the way out. In the reverse direction the mobile field was read from `F/quadrature_*.txt` through a column map whose source comment conceded it had never been checked against a real run.

6.2 What MoDELib3 actually offers

MoDELib3 does not need a scalar. `ClusterDynamicsFEM` already carries the immobile population as a finite-element *field* — `immobileClusters`, with `iSize = 8` components per node — and `ImmobileSinks` builds the sink strength from the local value at every quadrature point. `ClusterDynamics::output` writes that field, together with the `mSize = 4` mobile components, into the CD block of `evl/evl_<N>.txt`. Decisively, `ClusterDynamicsFEM::initializeConfiguration` reads it back:

```
if(size_t(configIO.cdMatrix().size())==mobileClusters.gSize()+immobileClusters.gSize())
{
    mobileClusters = configIO.cdMatrix().block(0,0,nNodes,mSize)...
    immobileClusters = configIO.cdMatrix().block(0,mSize,nNodes,iSize)...
}
```

The CD block is therefore a read/write field channel in both directions, on the node set the two codes already share. `evl/cdNodes.txt` gives the node coordinates in the same row order, so no search or interpolation is required.

6.3 The mapping

Table 8: CD block layout, `ClusterDynamicsParameters`: `mSize = 4`, `iSize = 8`. `immobileSpeciesVector = -1 1 1 1` makes family 0 the basal $\langle c \rangle$ vacancy family and families 1–3 the three prism $\langle a \rangle$ variants.

Columns	Block	Components	Units
0–3	mobile	C_v, C_i, C_{2i}, C_{3i}	atom fraction
4–7	immobile	$n_c, n_{a1}, n_{a2}, n_{a3}$	per b^3
8–11	immobile	$c_c, c_{a1}, c_{a2}, c_{a3}$	atom fraction

MoDELib works in Burgers-vector units, so its nodal number density is per b^3 while the 0-D loop density is an atom fraction. `ImmobileSinks` floors them as $n_{\text{floor}} = C_{\text{floor}}/\omega$ and $c_{\text{floor}} = C_{\text{floor}}$, which fixes the conversion:

$$n_{\text{MoDELib}} = \frac{C_{0D}}{\omega}, \quad c_{\text{MoDELib}} = c_{0D}, \quad \omega = \frac{\text{atomicVolume_SI}}{b_SI^3} = 0.355111 \, b^3. \quad (2)$$

The 0-D aligned/non-aligned pair is collapsed and the $\langle a \rangle$ total distributed over the three prism variants by the resolved-stress weights — the same Option-A reduction `modelib_export` performs, but applied *pointwise* rather than to a volume average. The mobile direction is a pure slice: MoDELlib’s four mobile species *are* the 0-D mobile species, in the same order and units.

6.4 Verification on the 3-D verification case

Tested against MoDELlib3/tutorials/zrmicro_coupled, the 1 μm single-crystal case with 24 115 CD nodes, at `evl_30.txt`.

Table 9: Field-handoff verification, 24 115 CD nodes.

Check	Result
CD block rewrite round-trip	identical, $24\,115 \times 12$; header and all dislocation records preserved
0-D $\rightarrow$ MoDELlib $\rightarrow$ 0-D immobile	max relative error 4.47×10^{-16} (machine precision)
Mobile field read back	C_v spans 8.46×10^{-15} to 6.70×10^{-6} , a dynamic range of 7.9×10^8
Spatial structure	$\text{corr}(\log_{10} C_v, \text{distance to nearest face}) = +0.63$
March on the real node set	24 115 nodes over 1 dpa in 9.5 s, 0 failures
Field written back	n_{a1} spans a factor 4.4 across the domain

The information content of the handoff rises from 4 numbers to $24\,115 \times 12 = 289\,380$ numbers, and the boundary-layer structure that the Dirichlet condition imposes on the mobile field — eight decades in C_v , correlating with distance to the surface — now reaches the slow march instead of being averaged away.

A precision detail. Restoring the aligned/non-aligned split on the return trip initially lost eight significant digits on the minor partner. The fractions were formed as f and $1 - f$; when the pair is lopsided (and $C_{iL} \gg C_{aiL}$ is normal at low dose) f rounds to $1 - \epsilon$ and $1 - f$ retains only the bits surviving the cancellation. Forming both fractions directly from the pair restores full precision, giving the 4.47×10^{-16} of Table 9.

6.5 Notebook wiring

`coupled_0d_3d_ZrMicro.ipynb` gains a "field" fast-solve backend, selected automatically when a CD field is present. In that mode the march runs on the CD node set rather than the synthetic 1-D line, seeds the immobile state from the 3-D field itself, and after each dose interval writes `evl/evl_coupled_<dose>dpa.txt` — a complete configuration, with the dislocation records copied through unchanged, that MoDELlib can be restarted from. The previous scalar path and the analytic placeholder both remain available and unmodified.

Because MoDELlib reads its configuration at start-up, the handback takes effect on a *restart* from the written file. That is the rhythm the operator split requires anyway (fast solve $\rightarrow$ slow march $\rightarrow$ fast solve), but it does mean the coupling is restart-driven rather than in-process: at the default six snapshots that is six DDomp launches, which is unremarkable; at hundreds of dose intervals it would argue for driving MoDELlib through `pyMoDELlib` instead.

6.6 Measured cost of the field upgrade

The comparison needs care, because the old path never ran on the mesh at all. `RUN["n_points"] = 25`: it marched a synthetic 1-D line of 25 points bearing no relation to the geometry, while the field path marches the 24 115 CD nodes — 965 times as many. A same-work control is therefore

included: the git-HEAD solver driven over the *same* 24 115 nodes, which is what the old solver would have cost to do the new job.

Test. The notebook’s default schedule — 1 to 26 dpa, five intervals, `substeps_per_interval = 20`, i.e. 100 batch subprocesses — run to completion in all three configurations, seeded at 1 dpa from the 0-D reference and, for the field configurations, from the CD field itself.

Table 10: Full 1–26 dpa coupling march, 100 batch subprocesses, 24 cores.

	Configuration	q	wall [s]	s/substep	ms/point/substep
A	Old as configured (git HEAD, 1-D line)	25	4.3	0.04	1.715
B	Old solver at the new workload	24 115	1117.4	11.17	0.463
C	New: $9 \times 9 + \text{AD} + \text{reuse}$	24 115	574.2	5.74	0.238

Two distinct readings, and both are needed to avoid misrepresenting the change:

- **Same work, B versus C: 1.95× faster.** That is the honest solver speedup — identical points, identical schedule. It exceeds the 1.67× measured at $q = 1024$ in Table 7 because the per-case solver construction the old binary pays scales with q while the new binary’s per-thread construction does not: 24 115 solver builds per substep become 24.
- **Different work, A versus C: 134× more wall clock for 965× the points.** The old configuration is faster in absolute terms only because it was doing 1/965 of the work and emitting eight numbers. Per point per substep the new path is 7.2× cheaper ($1.715 \rightarrow 0.238$ ms), part solver improvement and part fixed process-and-parse overhead amortizing over far more cases.

For scale: the 3-D solve itself (`clean_run.sh` → DDomp) takes hours. Against that, neither 4 s nor 10 min makes the march the pacing item of a coupled run.

Consistency. Over the full 1–26 dpa march, configurations B and C agree to 9.05×10^{-7} in the species. That is *tighter* than the 8.2×10^{-4} measured after the first 5 dpa interval alone, and the direction is informative: the early transient difference between two step sequences washes out rather than accumulating, because the loop populations are approaching a quasi-steady state and the dynamics are attracting. The largest disagreements sit at the stiff near-boundary nodes, where the frozen mobile field is eight decades below the interior value. It is a tolerance effect, not a change of answer.

7 Integrator comparison: MoDELib’s nodal update versus the CVODE march

The two codes advance the *same* immobile physics — eight components per node — with entirely different schemes. This section characterizes both, since the difference is a systematic source of 0-D/3-D discrepancy that is independent of the physics non-correspondences already documented in `CLAUDE.md`.

Everything said here about “MoDELib” means MoDELib3, the standalone route actually used in this work. The upstream `MoDELib-fullCD` discretizes the same physics differently again — a Galerkin mass-matrix projection of the rate, then explicit Euler and sequential implicit loss factors, one step per dose step rather than twenty — and is characterized separately in Section 8.1.

7.1 What MoDELib actually does

It is worth stating precisely, because the scheme is often described as explicit Euler and it is not. `ClusterDynamicsFEM::solveImmobileClusters` loops over nodes and, inside each node, over `nSub` = 20 substeps of $dt = dtMax/20$, applying

```
// implicit for the linear losses, explicit for the sources
const double nNew((n(k)+dt*nucRate)/(1.0+dt*lossN));
const double cNew((c(k)+dt*(gdot(k)+Gk+clusShare(k)*clusC(pol)-contentSink))/(1.0+dt*lossC));
```

This is a first-order **semi-implicit (IMEX)** update: the sources (nucleation, absorbed flux, cascade seed) are explicit, while the linear loss coefficients — coalescence, $1/\tau_{vL}$, and the thermal-emission rate recast as a first-order coefficient — sit in the denominator. The source comment records the reason: “ τ_{vL} is short compared with a dose step ... an explicit update would give $n(1 - dt/\tau) < 0$ and collapse the vacancy-loop density.”

With $G = 10^{-7}$ dpa/s and `dtMax` = 6.95868964e19 in MoDELib units (time unit 1.4371×10^{-13} s), one dose step is 10^7 s and the immobile substep is

$$dt_{\text{MoDELib}} = \frac{10^7 \text{ s}}{20} = 5 \times 10^5 \text{ s.} \quad (3)$$

7.2 Test method

Three instruments, all exercising the *same* `rate_equations_core.h` right-hand side so that no model difference contaminates the comparison:

1. **Spectrum.** The frozen-mobile 9×9 immobile Jacobian is formed at states sampled along a converged trajectory and its eigenvalues taken. The forward-Euler stability limit is $dt < 2/|\text{Re } \lambda|_{\max}$.
2. **Fixed-step explicit reference.** A `-euler_nsub=N` mode was added to the solver: N forward-Euler substeps per output interval on the same core, with the per-substep positivity clamp MoDELib applies. This is the *control* — what the update would do without the implicit loss treatment — not a model of MoDELib’s scheme.
3. **Reference solution.** CVODE at `rtol=1e-12`, `atol=1e-26` with the AD Jacobian, against which every other result is measured.

Two nodes of the 1 μm case bracket the range: the interior node of maximum C_v and the near-boundary node of minimum C_v .

7.3 Stability

Table 11: Spectrum of the frozen-mobile 9×9 immobile Jacobian.

Dose [dpa]	$ \text{Re } \lambda _{\max} [\text{s}^{-1}]$	$dt_{\text{stab}} = 2/ \text{Re } \lambda _{\max}$	stiffness ratio	$dt_{\text{MoDELib}}/dt_{\text{stab}}$
0.99	1.854×10^{-1}	10.8 s	1.5×10^5	46 338
2.96	1.853×10^{-1}	10.8 s	5.7×10^7	46 335
5.91	1.853×10^{-1}	10.8 s	4.0×10^5	46 335
11.16	1.853×10^{-1}	10.8 s	1.2×10^5	46 335
15.78	1.853×10^{-1}	10.8 s	3.1×10^4	46 335
21.05	1.853×10^{-1}	10.8 s	1.8×10^7	46 335
26.52	1.853×10^{-1}	10.8 s	9.9×10^4	46 335

The fast eigenvalue is essentially dose-independent at 0.185 s^{-1} — the vacancy-loop dissolution channel, $n_{vL,\text{nuc}}/\tau_{vL}$ — so a forward-Euler step must satisfy $dt < 10.8 \text{ s}$. MoDELib runs at $5 \times 10^5 \text{ s}$, **46 000 times the explicit limit**. Its implicit loss treatment is therefore not a refinement; it is what makes the scheme usable at all.

Table 12 shows the consequence directly, from the fixed-step explicit control at MoDELib’s own substep count.

Table 12: One 1 dpa step at the near-boundary node: a genuine forward Euler at $dt = 5 \times 10^5 \text{ s}$ annihilates the entire $\langle c \rangle$ vacancy-loop family, exactly the failure the MoDELib source comment anticipates. The $\langle a \rangle$ families, which carry no fast decay channel, are unaffected.

Species	initial	CVODE reference	explicit Euler, $N = 20$	rel. error
C_{iL}	2.53×10^{-3}	5.69×10^{-3}	5.69×10^{-3}	1.8×10^{-11}
C_{aiL}	1.60×10^{-3}	3.44×10^{-3}	3.44×10^{-3}	2.9×10^{-11}
$C_{iL,i}$	2.16×10^{-1}	2.79×10^{-1}	2.79×10^{-1}	2.2×10^{-10}
$C_{aiL,i}$	1.37×10^{-1}	1.73×10^{-1}	1.73×10^{-1}	2.3×10^{-10}
C_{vL}	3.48×10^{-10}	3.48×10^{-10}	1.0×10^{-20} (floor)	1.00
C_{avL}	2.02×10^{-10}	2.02×10^{-10}	1.0×10^{-20} (floor)	1.00
$C_{vL,v}$	7.21×10^{-4}	7.21×10^{-4}	1.0×10^{-20} (floor)	1.00
$C_{avL,v}$	4.20×10^{-4}	4.20×10^{-4}	1.0×10^{-20} (floor)	1.00

Both schemes actually in use are unconditionally stable on the stiff channel, but by different mechanisms. MoDELib’s amplification factor $1/(1 + dt\lambda) \in (0, 1)$ for every $dt > 0$ and $\lambda > 0$: L-stable on the linear part, and positivity-preserving by construction. CVODE BDF is A-stable at orders 1–2 and stiffly stable to order 5, and in addition selects its own step from a local error estimate.

7.4 Accuracy and speed

Table 13: One 1 dpa dose step, interior node ($C_v = 6.70 \times 10^{-6}$), against the `rtol=1e-12` reference. “core evals” counts evaluations of the shared right-hand side.

Integrator	substeps	dt [s]	core evals	max rel. error	t [ms]
Explicit Euler (MoDELib’s <code>nSub</code>)	20	5.0×10^5	20	9.27×10^4	0.14
Explicit Euler	200	5.0×10^4	200	9.27×10^3	0.27
Explicit Euler	2000	5.0×10^3	2000	9.26×10^2	1.18
Explicit Euler	20000	5.0×10^2	20000	9.17×10^1	13.71
CVODE BDF $9 \times 9 + \text{AD}$, <code>rtol=1e-6</code>	161 (adaptive)	6.2×10^4	366	2.10×10^{-6}	1.50

At the near-boundary node CVODE needs only 37 steps and 84 core evaluations for 1.27×10^{-8} , the frozen mobile field there being effectively zero.

The explicit control converges cleanly at first order — a factor 10 in error per factor 10 in step count — which confirms the order; the enormous constant reflects that it is simultaneously unstable.

What can and cannot be claimed for MoDELib’s scheme. Its IMEX splitting was not reimplemented on the ZrMicro equations, so no error *number* is quoted for it here. What is structural, and visible in the code itself, is that it is **first-order accurate with no error control**: no local error estimate, no step rejection, no adaptivity, and 20 fixed substeps per dose

step whatever the local stiffness. Against that, CVODE selects its own order (up to 5) and step from a local error estimate at `rtol`= 10^{-6} .

Speed. Per node per dose step, MoDELib performs 20 nodal updates of eight components with no Jacobian, no linear solve and no error estimate; the comparable explicit control costs 0.14 ms against CVODE’s 1.50 ms interior and 0.76 ms near the boundary. So MoDELib’s immobile update is roughly **an order of magnitude cheaper per node**, which is precisely the price of first order and no adaptivity. At the scale of the coupled march this is the difference between the 3-D solve’s nodal update being free and the 0-D march costing the 574 s of Table 10.

7.5 Consequence for the coupling

Table 14: The two integrators side by side.

	MoDELib <code>solveImmobileClusters</code>	ZrMicro coupled march
Scheme	semi-implicit (IMEX), nodal	CVODE BDF, adaptive
Order	1	up to 5
Step	fixed, 5×10^5 s (20/dpa)	adaptive (~ 161 /dpa interior)
Error control	none	local estimate, <code>rtol</code> = 10^{-6}
Stability	L-stable on the linear losses	stiffly stable to order 5
Linear algebra	none (diagonal by construction)	dense 9×9 LU, exact AD Jacobian
Cost per node	~ 0.14 ms / dpa	~ 0.8 – 1.5 ms / dpa
Accuracy	first order, unquantified	2×10^{-6} measured

The operator split is itself first order in $\Delta\gamma$ (Section 7 notwithstanding, this is the scheme’s design order). MoDELib’s immobile update is therefore *consistent with* the splitting order, and CVODE’s six additional digits on the slow march buy nothing unless $\Delta\gamma$ is refined alongside them.

Where the difference does matter is the return path. Once the marched field is written back through the CD block (Section 6.6), the two schemes’ different truncation errors become a systematic 0-D/3-D discrepancy — a *fourth* candidate alongside the three physics non-correspondences listed in `CLAUDE.md` (the DAD bias reconciliation, the bi-pyramid vacancy family, and size-dependent vacancy-loop emission). Anyone chasing a residual disagreement between the interior 0-D state and the 3-D result should rule this one out first, because it is the cheapest to test: raise `nSub` in `solveImmobileClusters` and see whether the disagreement moves.

8 Provenance: how the two schemes came to be

Section 7 compared the two integrators as they stand. This section records where each came from, because the histories explain the asymmetry between them and carry a maintenance consequence.

Correction: the upstream is MoDELib-fullCD, not MoDELib2-NNL. An earlier revision of this section took MoDELib2-NNL as the ancestor of MoDELib3’s cluster dynamics and concluded, from that repository, that “the immobile field does not exist at all upstream”. Every factual claim made about that repository holds — a checkout confirms `iSize = 0`, an empty `solveImmobileClusters()` body, no `ImmobileSinkRate.h` or `SpatialODESolver.h`, and the literal `// Missing immobile sinks placeholder at ClusterDynamicsFEM.cpp:110` — but the baseline was the wrong one, so the conclusion drawn from it does not follow. The CD-complete upstream is [m1m335/MoDELib-fullCD](#) (commit 62ac34f), which carries a full immobile field, its own immobile integrator, and the loop sink on the mobile equation. Everything below has been re-derived from

a checkout of that repository, and the conclusions change: the MoDELib3 immobile solver is a *re-discretization* of an existing upstream scheme, not an invention from nothing.

The lineage is checkable in the parameter set. Of the cluster-dynamics keys read by `ClusterDynamicsParameters` 31 are common to both codes verbatim, and most of the remainder are renames of one another:

MoDELib-fullCD	MoDELib3 (Zr3d_ghoniem)
<code>loopAnnealTau0_SI</code> , <code>loopAnnealEa_eV</code>	<code>tau0_vLoop_SI</code> , <code>Ea_vLoop_eV</code>
<code>loopNucDefects</code>	<code>nNuc</code>
<code>loopCoalLL</code> , <code>loopCoalLN</code>	<code>cLL</code> , <code>cLN</code>
<code>loopCoalKappaLL</code> , <code>loopCoalKappaLN</code>	<code>kappaLL</code> , <code>kappaLN</code>
<code>loopCoalNetwork_SI</code>	<code>rhoNetwork_SI</code>
<code>loopCascadeFraction</code>	<code>loopCascadeFractions</code>
<code>minimumLoopSize</code>	<code>r_min</code>
<code>initLoopSinks_SI</code>	<code>initloopSinks_SI</code>

What is genuinely *new* in MoDELib3 is small and specific: `dadAnisotropy` and `dadZ0` (the DAD reconciliation with the 0-D), `atomicVolume_SI` (the CD atomic volume decoupled from the lattice cell volume), `concentrationFloor` (ZrMicro’s C_{floor}) and `loopSinkScale`. What is genuinely *lost* is `clusterDiscretizationTime/clusterDiscretizationFactor` and with them `initializeDiscreteClimbLoops()`: `fullCD` converts its continuum loop field into discrete climb loops once the dose reaches a set threshold, and MoDELib3 has no such transition at all. That capability is what the name “fullCD” denotes, and dropping it is the largest functional regression of the fork.

Table 15: Immobile-species integration across the three code states. The upstream column is MoDELib-fullCD, read from a checkout, not inferred.

	MoDELib-fullCD (upstream)	MoDELib3 (standalone route)	ZrMicro coupled march (this work)
iSize (immobile dofs/node)	8	8	—
Rate discretization	Galerkin: consistent-mass L^2 projection, CG to 10^{-4}	nodal collocation, no mass matrix	pointwise, per node
Time update	explicit Euler on the projected rate, then sequential implicit loss factors	one fused semi-implicit update	CVODE BDF
Order	1	1	up to 5
Step control	one step of <code>dtMax</code>	fixed, 20 substeps/dpa	adaptive
Error control	none	none	$\text{rtol} = 10^{-6}$
Loop sink on the mobile	<code>FirstOrderReaction</code>	<code>ImmobileSinks</code> (re-named)	frozen $\mathbf{C}_M^*$
Continuum $\rightarrow$ discrete loops	<code>initializeDiscreteClimbLoops</code>	<code>absent</code>	n/a

8.1 The upstream scheme, and how MoDELib3 re-discretized it

`fullCD` advances the immobile field in three distinct stages per DD step.

1. **Rate, by Galerkin projection.** `ImmobileSinkRate` is an `EvalFunction` evaluated at *quadrature points* — absorption growth, cascade nucleation, and homogeneous cluster nu-

creation from $-\frac{1}{2} \mathbf{C}^\top \mathbf{R}_2 \mathbf{C}$ — assembled into the linear form $(\text{test}(c_I), \dot{c}_I) dV$ and then *projected* onto the trial space by solving the consistent mass matrix $\mathbf{M} = (\text{test}(c_I), c_I) dV$ with conjugate gradients to 10^{-4} (`SpatialODESolver`). The stored rate is $\mathbf{M}^{-1} \mathbf{b}$.

2. **Explicit Euler.** `updateImmobileClusters(dt)` does $\mathbf{c}_I \leftarrow \mathbf{c}_I + \dot{\mathbf{c}}_I dt$ with $dt = dt_{\text{Max}}$ — one step per DD step, no sub-cycling.
3. **Implicit loss corrections, applied in sequence.** `applyImmobilePopulationCorrections` then divides the result by $(1 + dt \lambda)$ once per loss channel, in a fixed order: thermal annealing (with the content debited by $n_{\text{nuc}}(N_{\text{old}} - N_{\text{new}})$), content removal, then loop-loop and loop-network coalescence with their Avrami factors. A clamp enforces $N \geq 0$ and a minimum loop content $c \geq n_{\text{min}} \Omega N$.

MoDELib3 keeps the physics of stages 1 and 3 and changes how both are discretized. The rate is evaluated **at nodes** rather than projected, so the mass matrix and its CG solve disappear; and the explicit step and the loss divisions are **fused into a single semi-implicit formula**, $n^{\text{new}} = (n + dt \dot{n}_{\text{nuc}}) / (1 + dt \lambda_{\text{tot}})$, sub-cycled 20 times per dose step.

Neither choice is strictly better, and the trade is worth stating. The Galerkin projection is the more accurate representation of a smooth field, because it respects the element shape functions instead of sampling them at nodes; but it couples neighbouring nodes, and on a sharp gradient it overshoots. `fullCD` says so in its own comment — the corrected sink value exists “to adjust for negative values at the barycenter of the quadratic elements where we have a sharp gradient of sink values between two end nodes”. Nodal collocation cannot produce that artifact, at the cost of ignoring the sub-element variation entirely. Given that the 1 μm case has a boundary layer spanning twelve orders of magnitude in C_v (Table 21), sharp gradients are the normal condition here rather than the exception, and MoDELib3’s choice is defensible on those grounds — though it was not, so far as the change log records, made for that reason.

The sequencing also differs in a way that matters for large dt . `fullCD` applies its loss factors *one after another*, so with three active channels the result is $\prod_k (1 + dt \lambda_k)^{-1}$, whereas MoDELib3’s fused form gives $(1 + dt \sum_k \lambda_k)^{-1}$. The two agree to $O(dt^2)$ and diverge when any $dt \lambda_k \gtrsim 1$, which — given the measured $dt \lambda \approx 46\,000$ of Section 7 — is the operating regime, not a corner case. Both remain unconditionally positive, which is the property that matters most here; neither is a controlled approximation at that step size.

8.2 How MoDELib3’s variant reached its present form

MoDELib3 re-derived its update rather than porting `fullCD`’s, and in doing so it re-encountered the stiffness that `fullCD`’s loss factors already handle. It began fully explicit and reached the semi-implicit update through four documented corrections, recorded in `MoDELib3/ZR3D_GHONIEM_CHANGES.md`:

Issue 7 is worth singling out because its diagnosis was made the right way: the authors confirmed it was integration and not physics by checking the analytic balance $n^* = \dot{n}_{\text{nuc}} \tau_{vL} = 1.79 \times 10^{-8}$ against the 0-D $C_{vL} + C_{avL} = 1.78 \times 10^{-8}$, then verified the fix reproduced 1.79161×10^{-8} .

8.3 The coupled march scheme: outward from the 0-D solver

The march scheme has the opposite history. It did not have to discover stiffness, because it inherited a stiff integrator: `ZrMicro`’s `CVODE` BDF solver predates the coupling entirely, built for the standalone 0-D 19-equation model. The coupling was then added in three steps:

1. **freeze_mobile** — the operator split. The four mobile derivatives are zeroed *last*, after every reaction term has already read them as parameters, so the same right-hand side serves as the local immobile micro-model with an externally supplied $\mathbf{C}_M^*$.

Table 16: How the standalone immobile scheme reached its present form. Each stiffness failure was fixed by moving one more linear loss into the implicit denominator.

Issue	Symptom	Cause	Fix
7	$\langle c \rangle$ loops pinned on the positivity floor while $\langle a \rangle$ loops nucleated normally	τ_{vL} is $35\times$ shorter than the substep, so explicit Euler gave $n(1 - 35) < 0$, clamped every iteration	Integrate the linear losses implicitly; returns $n^* = \dot{n}_{\text{nuc}}\tau_{vL}$ exactly as $dt/\tau \rightarrow \infty$
8	Thermal emission could not be made implicit	It was written as an absolute rate, not proportional to the stored content	Recast as a first-order coefficient (divide by c) so it can enter the implicit update
14	$\langle a \rangle$ mean size 5042 at 1 dpa, then exactly $300.0 = n_{\text{nuc}}$ at 2 dpa — content collapsing $11\times$	The same stiffness failure in the coalescence channel left explicit: $\nu_{LN}\phi_{LN}dt_{\text{sub}} \approx 3.6$	Eq. (97) removes n and c proportionally, so both are first-order coefficients; moved implicit. $\langle a \rangle$ density went from $3\text{--}4\times$ the 0-D to within $\sim 30\%$
15	$\langle c \rangle$ content $350\times$ below the 0-D, mean size pinned at the nucleation size	Dissolution removed content proportional to the stored content	Release the birth content $n_{\text{nuc}}n/\tau_{vL}$ instead

2. **Pointwise dispatch** — because the split makes the slow march decoupled between points, each quadrature point becomes one case of the existing OpenMP `-batch_file` mode.
3. **This work** — accumulators moved to CVODES quadrature ($19 \rightarrow 9$), the exact AD Jacobian, and per-thread workspace reuse.

8.4 Three routes to the same conclusion

All three codes end up treating the immobile loss channels implicitly, and all three are right to, but they arrive from three different directions — and the agreement is stronger evidence than any one of them alone.

- **fullCD** put the losses in implicit denominators *by construction*: annealing, content removal and both coalescence channels are applied as $(1 + dt\lambda)^{-1}$ factors from the outset, while the source terms stay explicit. It is a deliberate IMEX split.
- **MoDELib3** re-derived the update, started fully explicit, and found the stiffness *empirically* — twice, as loop populations collapsing onto the positivity floor (issues 7 and 14) — then fixed it term by term until it had reconstructed the same partition fullCD already had.
- **The 0-D side** never exposed the problem at all, because BDF absorbed it silently from the start; it took the spectrum of Table 11 to put a number on what was being absorbed ($46\,000\times$ the forward-Euler limit).

That MoDELib3 rediscovered fullCD’s partition by symptom, one channel at a time, is the clearest single argument that the partition is forced by the physics and not a modelling preference. It is also an argument for reading the upstream: the two failures documented as issues 7 and 14 were both already answered by `applyImmobilePopulationCorrections`.

That difference in route leaves a difference in robustness, and it is the practical point of this section. MoDELib3’s scheme is unconditionally stable on the losses that *have been* moved into `lossN/lossC`. It offers no protection for one that has not. A newly added immobile loss term will be stable only if it is first recast as a first-order coefficient and placed in the denominator; written as an explicit rate it will reproduce issue 7 or issue 14, and — with no local error estimate and no step rejection — the only symptom will again be a species quietly pinned on the floor. CVODE would have flagged the same term through step rejection and error-test failures without anyone having to notice a collapsed population.

Sourcing. The upstream statements in this section were read directly from a checkout of `mlm335/MoDELib-fullCD` at commit `62ac34f` (`src/ClusterDynamics/`, `include/ClusterDynamics/`, `Library/Materials/`). MoDELib3/ itself carries no independent git history, having been flattened from a submodule — the commits present are DisloCluster’s own — so the fork point cannot be recovered from the repository and the correspondence above is established by reading the two sources side by side rather than from history.

One earlier piece of evidence should be retired rather than repeated. `MoDELib3/Library/Materials/Zr4.txt` does contain zero immobile keys, and that fact is correct; the *inference* drawn from it — that the upstream standalone route therefore could not produce an $\langle a \rangle / \langle c \rangle$ microstructure at all — was wrong. `Zr4.txt` is simply an older material file that happens to be carried in this checkout. Upstream, `Zr4_Fitted.txt` and `Zr4_BMD19_nuc.txt` each carry the full sixteen-key immobile set (`immobileSpeciesVector`, `loopCascadeFraction`, `loopNucDefects`, `loopContentTau_SI`, `loopAnnealTau0_SI`, `loopCoal*`, `n_min`, `dislocationSinks_SI`). The absence of those keys from one file in this repository says nothing about the upstream code.

9 The three recommendations, implemented and measured

The three items at the head of the earlier recommendation list have been built and measured on the real field: $q = 24115$ CD nodes of the $1\ \mu\text{m}$ cube, one 1 dpa substep, seeded from the shared 0.1 dpa state with the mobile field taken from the 3-D solve.

9.1 Recommendation 1: how the accumulators are carried

The choice is now explicit, `-acc_mode`:

Mode	Treatment
0	In the implicit state <i>and</i> in the error test — the legacy layout.
1	CVODES quadrature variables. Smallest Newton block, but the quadrature right-hand side needs its own core evaluation at the accepted step point, and the memo meant to reuse the residual’s evaluation essentially never hits (Section 5.5).
2	In the implicit state, with a per-component <code>atol</code> large enough that their error weights vanish (<code>CVodeSVtolerances</code>). Bigger Newton block, but <i>zero</i> extra core evaluations.

The prediction of Section 5.5 is confirmed, and quantified. Mode 1 has the smallest Newton block by a wide margin — an order of magnitude fewer LU flops than mode 0 — yet the *largest* core-evaluation count of the three, because the quadrature adds roughly one full sweep per step. Mode 2 gives that LU saving back and wins where the cost actually is: **26% fewer core evaluations than the legacy layout and 47% fewer than the quadrature**, with the fewest steps of the three. Modes 1 and 2 agree with each other to 1.7×10^{-5} against mode 0, which is the expected consequence of removing the accumulators from the error test, not a disagreement between them.

Table 17: Accumulator handling, $q = 24\,115$, one 1 dpa substep, AD Jacobian. Errors are against mode 0.

Mode	n	wall [s]	cpu [s]	steps	core evals	n_{LU}	LU flops
0 in state, error-controlled	19	4.193	74.01	3 346 941	4 184 924	961 510	2.20×10^9
1 CVODES quadrature	9	3.402	49.84	2 508 983	5 838 440	584 867	1.42×10^8
2 in state, atol relaxed	15	3.314	49.06	2 397 987	3 077 945	507 161	5.71×10^8

Mode 2 is now the coupling default. Mode 0 remains the default for standalone 0-D runs, which stay bit-identical.

9.2 Recommendation 2: the base+delta case protocol

Every batch line used to repeat the full parameter set. A line beginning @BASE now carries the shared defaults once and each case line only its overrides; a file without a @BASE line still parses exactly as before, and the solver reports on `stderr` which protocol it used so a benchmark cannot silently measure the wrong one.

Table 18: Case-file protocol, $q = 24\,115$. The two rows of each pair differ only in how the same cases are serialized. The upper block is the seed step, where the immobile state is still uniform and only the four mobile values differ; the lower block is the realistic mid-march state, where all thirteen trajectory components differ per node.

State	Protocol	file [MB]	tokens/case	wall [s]
uniform seed	full-line	35.39	88	5.410
	base+delta	1.58	4	3.407
realistic mid-march	full-line	36.75	89	6.911
	base+delta	7.13	13	5.369

The honest figure is the lower block: $1.29\times$ end to end, with the case file shrinking $5\times$. The upper block would read $1.59\times$, but only because a uniform seed leaves just four values varying per case; quoting it as the general result would overstate the gain, since after the first substep the immobile state varies node to node and thirteen values differ.

9.3 Recommendation 3: deduplicating points before dispatch

Neighbouring nodes in a smooth field integrate nearly identical initial-value problems. `run_immobile_step` now optionally groups points whose trajectory-determining components (the four frozen mobile values, the eight immobile components and ρ_N — not the accumulators, which reset every substep) agree to a relative tolerance, integrates one representative per group and scatters the endpoint back.

Table 19: Deduplication, $q = 24\,115$, one 1 dpa substep. Errors are against no deduplication.

dedup_rtol	integrations	factor	wall [s]	max rel. error
off	24 115	1.00	8.964	—
10^{-10}	17 582	1.37	6.042	0 (exact)
10^{-8}	17 582	1.37	5.983	0 (exact)
10^{-6}	17 582	1.37	6.610	0 (exact)
10^{-4}	15 753	1.53	5.683	9.9×10^{-5}
10^{-2}	6 368	3.79	2.301	9.7×10^{-3}

This is well short of the 10^2 – 10^3 collapse the earlier note speculated about, and the reason is worth recording. Tolerances from 10^{-10} to 10^{-6} all find the *same* 17 582 distinct states: those 6 533 merges are **exact** duplicates, and the error is not merely small but identically zero. They are the Dirichlet-pinned boundary nodes, where the mobile field is clamped to the same value. Away from the boundary the field varies smoothly and continuously across eight decades, so genuinely distinct states stay distinct at any tolerance tight enough to be free.

Buying more requires accepting error: 10^{-4} gives $1.53\times$ for 10^{-4} relative error, and 10^{-2} gives $3.79\times$ for about 1%. At an integrator `rtol` of 10^{-6} the defensible setting is 10^{-8} , which is what the coupling uses — $1.37\times$ **fewer integrations at zero error**. The speculation that this would dominate everything else was wrong for this field, and the measurement says so.

9.4 Combined effect

Taken together on the coupling path — mode 2, base+delta, deduplication at 10^{-8} , on top of the workspace reuse and AD Jacobian of the earlier sections — the per-substep cost at $q = 24\,115$ falls from the 8.96 s of the unduplicated legacy-protocol path to 5.98 s, while the core-evaluation count drops 26% and the roundoff-warning pathology stays cured.

10 Two options for time integration by operator splitting

Everything in this section compares two ways of taking the *slow* step of the operator split. Both solve the same problem, by the same split, with the same fast step: at every substep the steady mobile field $C_M^*(x)$ is solved for the immobile state currently held — MoDELib3’s `ClusterDynamicsFEM::solveMobileClusters`, which has no time derivative at all — and then frozen while the immobile population is advanced. They differ only in how that advance is integrated.

	Name	Slow step
Option A	IMEX	Implicit-explicit Euler with Patankar stabilization: MoDELib3’s own nodal update $n \leftarrow \frac{n + \Delta t \dot{n}_{\text{prod}}}{1 + \Delta t \lambda_{\text{loss}}},$ production explicit, destruction implicit. The Patankar form is unconditionally positive for any step size and any non-negative state, because a positive numerator is divided by something greater than one. It is first order and carries no error estimate, so accuracy follows the step size alone (20 substeps per 1 dpa output step). Runs inside DDomp.
Option B	Adaptive	SUNDIALS CVODE, adaptive-order adaptive-step BDF, applied at every node with the mobile species frozen. Order (1–5) and step size are both selected from a local error estimate against <code>rtol/atol</code> , so the accuracy is requested rather than inferred. <code>acc_mode=2</code> , exact AD Jacobian, per-thread workspace reuse, base+delta protocol, deduplication at 10^{-8} . Driven from Python; the mobile field is re-solved every <code>FEM EVERY</code> substeps and frozen in between.

On the names. These were called *standalone* (Option A) and *coupled* (Option B) up to and including the seeding study, and every figure directory and table column written before this section used those words. They described how each option was *driven* rather than what it *is*, and “coupled” was actively misleading: both options are coupled 0-D $\leftrightarrow$ 3-D in the sense that matters, since both re-solve the mobile field against the evolving immobile state. The word “coupled” is therefore still correct for the *architecture* — the field handoff of Section 6.6 — and is kept there. It is no longer a route name. The canonical spellings live in `dislocluster_code/integration.py`, which every driver imports, so that directory names, table columns and plot legends cannot drift apart.

The measurements that follow were made on the 1 μm single crystal from *one* shared microstructure — the 0-D state at 0.1 dpa written uniformly onto all 24 115 CD nodes — advanced to 20.1 dpa in 5 dpa intervals. Section 10.7 then shows that the shared seed is not needed at all.

10.1 The fast step belongs inside the loop

The first version of this comparison did not perform an operator split at all, and the defect is worth stating plainly because it invalidated part of the earlier numbers.

MoDELib3’s `ClusterDynamicsFEM::solve()` *is* the split:

```
solveMobileClusters();      // FAST -- steady C_M*(x). No time derivative
                             // anywhere; the immobile population enters only
                             // through ImmobileSinks, evaluated at every
                             // quadrature point from the LOCAL state.
solveImmobileClusters();    // SLOW -- dt update of the nodal n_I, c_I
```

The Adaptive route was meant to keep the first call and replace the second. What it actually did was read the mobile field out of the *IMEX run’s* recorded snapshots. Under that arrangement C_M never responds to the immobile state the march is building: the coupling is one-way, and the march has no mechanism to relax a seed that is not already at quasi-steady state. Worse, the nearest recorded snapshot to the 0.1 dpa seed sits at 1.1 dpa, outside the 0.75 dpa acceptance window, so the entire first interval — a quarter of the march — ran on the raw uniform seed field.

The fix is a new DD.txt scalar, `useImmobileSolver` (default 1, so every pre-existing case is untouched). With `useImmobileSolver=0` a DDomp run performs the fast step and passes the immobile field straight through. The march then calls it as a subroutine: write the current per-node immobile state into the `ev1` CD block, run one DDomp step to obtain $C_M^*(x)$ for that state, freeze it, and integrate the immobile ODEs at every node. Because `runSingleStep` writes output *before* incrementing `runID`, a run with `startAtTimeStep=0` and `Nsteps=1` reads `ev1_0.txt` and overwrites the same file, which makes the exchange an in-place round trip. `dislocluster_code/coupling/qssa.py` drives it and asserts on DDomp’s immobile solver SKIPPED line, so a binary predating the flag fails loudly rather than advancing the immobile field twice.

10.2 A non-terminating mobile solve, and its cause

Closing the loop exposed a defect that the IMEX route had never triggered. Asked to solve the mobile field against an immobile state supplied by the march, `solveMobileClusters` does not converge: `cError` falls from 5.8×10^5 to 1.3×10^{-4} in nine iterations and then oscillates around 5×10^{-5} indefinitely. The loop is

```
while (cError > cTol)
```

with `cTol = 10-5`, *no iteration cap and no stagnation test* — inherited verbatim, since MoDELib2-NNL, MoDELib-fullCD and MoDELib3 all carry the identical loop. So DDomp never returns.

Cause. The one structural difference from upstream is the cause. fullCD has *no mobile clamp at all*; MoDELib3 inserted `clampMobileClusters()` between the Newton update and the error test, which makes the iteration a *projected* Newton method.

The obvious explanation — that the active set oscillates — is wrong, and an instrumented run refutes it: the set freezes at 7090 of 96 460 dofs by iteration 3 and reports zero entering and zero leaving on every iteration after. What actually fails is that the Newton matrix `dmBWF+bWF_R1+bWF_R2+bWF_RI` is the *unconstrained* Jacobian and does not know any dof is pinned. Each iteration it computes an increment for every dof, including moves for the pinned ones; the projection discards exactly those moves, but the increment for the *free* dofs was computed on the assumption that the pinned ones would move. The free dofs therefore respond, every iteration, to a coupling that is immediately cancelled, and that inconsistency does not decay. Solve-then-project is not a consistent method for a constrained system; the consistent form is an active-set method, in which the pinned rows are eliminated and the reduced system solved.

The intent is visible in the routine’s own comment — the clamp mirrors ZrMicro, “which applies $y = \max(y, C_{\text{floor}})$ before every *rate evaluation*”. In ZrMicro the floor belongs to the rate evaluation. Here it mutates the state inside the iteration, and only that breaks convergence.

Table 20: Mobile solve on an identical marched immobile state, 24 115 CD nodes. The last column is the maximum relative error in C_i against the converged field, over the 22 344 nodes above the floor.

Configuration	iters	wall [s]	converged	cError	max err. C_i
clamp in loop (as shipped)	30*	392	no	6.06×10^{-5}	53.6%
clamp deferred	5	112	yes	6.83×10^{-7}	—
clamp deferred, cTo1= 10^{-8}	6	120	yes	2.26×10^{-15}	0.0
damped $w = 0.5$	30*	400	no	9.42×10^{-5}	4.0%
damped $w = 0.7$	20	283	yes	8.21×10^{-6}	0.19%
error test excludes floored dofs	40*	492	no	4.67×10^{-5}	21.1%
cTo1 relaxed to 10^{-4}	19	258	“yes”	9.22×10^{-5}	49.9%

* stopped at the iteration cap.

Every remedy, measured on one saved state. Deferring the clamp restores **quadratic** convergence — $2.95 \times 10^4 \rightarrow 1.23 \times 10^3 \rightarrow 6.77 \times 10^1 \rightarrow 5.73 \times 10^{-2} \rightarrow 6.83 \times 10^{-7} \rightarrow 2.26 \times 10^{-15}$, each error the square of the last — and is the fastest option by a factor of 2.5.

Three of the alternatives are instructive failures. Changing the error metric is *bit-identical* to the baseline, all sixteen digits, iteration for iteration: a floored dof has $c^{\text{new}} = c^{\text{old}}$ and already contributes nothing to the step norm, so the free dofs are genuinely oscillating. Under-relaxation does converge, but only linearly, at roughly a factor of two per iteration. And relaxing cTo1 is actively dangerous: it reports success at iteration 19 while accepting a field that is 49.9% wrong in C_i at the worst node.

Verification. The chosen fix was checked four ways rather than assumed from the fact that it terminates. Convergence is quadratic; the fields at $c\text{To1} = 10^{-5}$ and 10^{-8} are *bit-identical*, so five iterations already reach machine precision; the independent damped route reaches the same field to within 0.19%, with the means agreeing to five figures, so the clamp was corrupting the iteration and not the answer; and the result is physical — zero negative entries and exactly the same 7090 floored dofs as the clamped runs, because the floor is still applied, once, after the loop.

The stalled iterate is not merely unrecognized as converged. Its means agree with the converged field to four figures, but two nodes exceed 1% and one reaches 53.6% in C_i . A handful of nodes at the clamp boundary drives the whole failure, and because the error test is a maximum over nodes, those few alone prevent it from ever settling.

What was changed. Five optional DD.txt scalars (`mobileSolverTolerance`, `mobileSolverMaxIterations`, `mobileSolverRelaxation`, `mobileSolverErrorMode`, `mobileSolverClampInLoop`), all defaulting to the historical behaviour except the iteration cap, which now retains the *best* iterate and so converts a hang into a bounded, reproducible result. The Adaptive fast solve sets `mobileSolverClampInLoop=0` in its own simulation directory and warns if any solve returns capped rather than converged. The IMEX route’s defaults are deliberately left unchanged: making the deferral global is very likely correct, but it alters a path that currently runs to completion, and that warrants a full IMEX verification first.

10.3 Does the split repair a seed that is not quasi-steady?

This is the question the arrangement exists to answer, and it can be asked directly: solve the fast step once against the seed’s own immobile state, before any immobile marching, and compare with what was seeded.

Table 21: The 0.1 dpa 0-D seed against the quasi-steady field solved for that same immobile state. 24 115 CD nodes, 573 K, 10^{-7} dpa/s. Reproduced to the digit on an independent invocation.

Species	seeded (uniform)	QSSA mean	seed / QSSA	QSSA min	QSSA max
C_v	1.134×10^{-5}	1.643×10^{-8}	690	1.0×10^{-20}	6.15×10^{-8}
C_i	1.645×10^{-10}	1.754×10^{-12}	94	4.1×10^{-27}	6.77×10^{-12}
C_{2i}	1.113×10^{-12}	8.966×10^{-15}	124	3.4×10^{-53}	3.46×10^{-14}
C_{3i}	4.455×10^{-13}	5.977×10^{-15}	75	1.3×10^{-70}	2.31×10^{-14}

The seed was very far from quasi-steady, and the fast step supplies the whole difference. Two corrections are happening at once, and they are worth separating:

- **Magnitude.** The seeded C_v is $690\times$ the quasi-steady mean. The 0-D reaches its own steady balance against a homogenized sink strength; the 3-D domain has, in addition, six Dirichlet faces holding the mobile species at thermal equilibrium and draining the interior.
- **Structure.** The seed is uniform by construction, so its spatial contrast is exactly 1. The relaxed C_v field runs from 10^{-20} at the faces to 6.15×10^{-8} in the interior — twelve orders of magnitude. The whole boundary layer is absent from the seed and is created by the fast solve. No amount of immobile marching could have produced it, because the immobile equations carry no spatial derivative at all.

The C_i minimum of 4.1×10^{-27} is the imposed boundary value $\exp(-3.0 \text{ eV}/kT)$, deliberately *not* floored at 10^{-20} : flooring it would raise the boundary concentration by seven orders and quietly disable the grain boundary as an interstitial sink, which is the one piece of physics the 3-D model adds over the 0-D.

So the answer is yes. The split reaches quasi-steady state on its own, and it does not need the 0-D seed to be right — which is fortunate, because a 0-D seed can never be right about a boundary layer it does not represent.

10.4 A calibration error that voids the earlier physics numbers

Every loop density and mobile concentration reported in earlier revisions of this section is void. The cause is not in the coupling or in the solvers; it is that the driver built its 0-D model straight from the input workbook.

The workbook is *not* the calibrated model. Section 6c of `coupled_0d_3d_ZrMicro.ipynb` says so explicitly — 28 parameters have drifted from the fitted set and 11 are absent from the workbook entirely — and pins every fitted value before building the model chain. `run_coupled_vs_standalone.py`

and the seed builder both skipped that step, so they integrated a different model. Integrated to 20 dpa at 573 K and 10^{-7} dpa/s:

0-D parameter set	N_a	c_a	C_v
workbook only (what the marches used)	2.32×10^{-2}	8.77	4.2×10^{-10}
calibrated	8.05×10^{-8}	1.34×10^{-4}	5.01×10^{-6}

Five orders of magnitude in loop density. c_a is an atom fraction, so the uncalibrated 8.77 is impossible on its face; the calibrated N_a is $6.7 \times 10^{21} \text{ m}^{-3}$ with a mean radius of 4.4 nm, which is where irradiated Zr should be. The fitted set is now carried by `disloccluster_code/zerod/calibration.py` and both drivers build through it.

Two conclusions that were drawn from the uncalibrated runs are withdrawn. The $\langle a \rangle$ content was reported as reaching a value greater than unit atom fraction, which is impossible; with the calibrated set it does not. The $\langle a \rangle$ *number density* does still grow without bound when averaged over the whole domain, but for a reason that has nothing to do with the parameter set — see Section 10.5, where it is localized to the Dirichlet faces. And the 3-D mobile field was reported as sitting some $180\times$ below the 0-D, which was read as either grain-boundary physics or a scaling error in the sink term. It was neither: with the calibrated set the interior plateau of the fast solve is 4.85×10^{-6} against the 0-D’s 5.01×10^{-6} — agreement to 3% — with the boundary layer falling to the thermal-equilibrium value $C_v^{\text{eq}} = 8.45 \times 10^{-15}$ at the faces. The well-mixed limit is satisfied, and no separate test of it was needed.

What is unaffected. The mobile-solver defect and its fix (Section 10.2) are pure numerics: the iteration either converges or it does not, for any parameter set, and the measurements there were made on a single saved state with all configurations sharing it. The 0-D solver itself was also checked and is not implicated — built from the pre-rewrite commit `acb884d` and from HEAD, it produces identical output to every printed digit, and it reproduces the experimental reference run (`output/20260806_090557_15c465f_coupled0d3d`) to four significant figures in the loop content at 0.1, 0.5 and 1 dpa.

A comparison to make carefully. That reference directory contains both a pure 0-D run and a `march_state.npz` from an Adaptive march with the placeholder fast solve. They are different objects, and comparing a free 0-D integration against the marched states — which this work did twice before noticing — looks like a bug and is not one. Freezing the mobile field holds the nucleation driver C_{3i} at its frozen value, so the marched $\langle a \rangle$ density grows to 3.63×10^{-7} at 1 dpa where the free 0-D saturates at 8.05×10^{-8} . That $4.5\times$ gap is the operator-splitting signature itself, present in the old placeholder run as well.

10.5 The whole-domain mean is the wrong statistic

With the calibrated set in place, the Adaptive march was run from a seed at 1 dpa — past the mobile transient — to 21 dpa in 5 dpa intervals, with the fast step re-solved every fifth immobile substep. Sixteen fast solves, every one converged, none capped. Averaged over all 24 115 nodes the result looks alarming:

Dose [dpa]	Quantity	0-D	Adaptive, all nodes	ratio
6	N_a	8.051×10^{-8}	9.600×10^{-6}	119
11	N_a	8.051×10^{-8}	1.913×10^{-5}	238
16	N_a	8.051×10^{-8}	2.866×10^{-5}	356
21	N_a	8.051×10^{-8}	3.819×10^{-5}	474
21	N_c	1.780×10^{-8}	1.786×10^{-8}	1.003
21	c_c	3.077×10^{-3}	2.977×10^{-3}	0.967

The $\langle c \rangle$ family tracks the 0-D to better than 4%. Only $\langle a \rangle$ diverges, and it does so *exactly* linearly: the increments are 9.53×10^{-6} per 5 dpa, equal to four figures. That is the signature of a constant source with no sink whatever, not of an instability.

It is confined to the Dirichlet faces. Binning the nodes by fractional distance to the nearest face, at 21 dpa:

Shell	Nodes	N_a mean	c_a/N_a	C_i mean	share of $\sum N_a$
surface (< 0.02)	6522	1.409×10^{-4}	300.8	4.11×10^{-27}	0.998
0.02–0.10	4371	2.002×10^{-7}	1388	1.79×10^{-10}	0.001
0.10–0.30	6546	6.060×10^{-8}	1838	5.86×10^{-10}	0.000
0.30–0.60	5385	5.946×10^{-8}	1864	5.92×10^{-10}	0.000
core (> 0.60)	1291	6.005×10^{-8}	1860	5.89×10^{-10}	0.000

The 6522 surface nodes — 27% of the node set, because second-order elements place that many on the six faces — carry 99.8% of the total. Their c_a/N_a is 300.8 and `n_iL_nuc` is 300: every loop there is a fresh nucleus that has never absorbed a single interstitial.

The mechanism is the one already recorded for MoDELlib3’s own IMEX run. Cascade nucleation is proportional to dose rate and fires uniformly in space. Both loop growth and the only removal channel, coalescence, are driven by the absorbed mobile flux — and the Dirichlet condition pins C_i to 4×10^{-27} on the faces. The boundary is a sink for mobile defects but *not* for loops, so loops are created there and nothing ever happens to them again. That the Adaptive march reproduces the artifact is evidence for the coupling, not against it: the same boundary physics emerges whether MoDELlib3’s nodal scheme or ZrMicro’s CVODE march advances the immobile population.

IMEX does the same thing, to four figures. The claim above is only worth making if MoDELlib3’s own immobile solver produces the artifact too. Binning the IMEX run — same mesh, same seed, same fast step, but its nodal semi-implicit scheme in place of the CVODE march — at 6 dpa:

At 6 dpa	Adaptive march	IMEX
whole-domain mean N_a	9.5996×10^{-6}	9.5994×10^{-6}
surface shell N_a	3.529×10^{-5}	3.528×10^{-5}
surface share of $\sum N_a$	0.994	0.994
surface c_a/N_a	303.1	303.1
core N_a	6.007×10^{-8}	6.338×10^{-8}
core c_a/N_a	1860.1	1808.3

Two integrators as different as CVODE BDF under error control and a first-order nodal semi-implicit update agree on the boundary population to four significant figures. That is the expected outcome rather than a coincidence: at the faces the only surviving term is cascade nucleation at a constant rate, which is linear in time, and both schemes integrate a constant exactly. The routes disagree by 5% where the physics is actually nonlinear — in the interior. The artifact is therefore a property of the boundary conditions and the loop model, shared by both codes, and not something the coupling introduces.

The interior is well behaved. Per node, the growth factor of N_a over 1–21 dpa has median 0.746, and the entire p75–p100 band is a single repeated value, 1749.9, shared by every surface node — the distribution is two spikes, not a tail. Interior N_a settles at 6.0×10^{-8} against the 0-D’s 8.05×10^{-8} and is flat from 6 dpa onward, with $c_a/N_a = 1860$ against the 0-D’s 1661.

Consequence for every comparison in this section. A whole-domain mean over this geometry is a measurement of the boundary shell, not of the material. All route-to-route comparisons below therefore report the *interior* mean — the innermost quartile by distance to the nearest face — with the whole-domain figure retained only in the machine-readable output. This is not a cosmetic choice: it is the difference between a ratio of 474 and a ratio of 0.75.

10.6 Does the seed dose matter? Seeding inside and outside the transient

The Adaptive march freezes the mobile field for FEM EVERY immobile substeps at a time, while the 0-D is a genuinely adaptive integration that resolves the mobile transient with whatever step size that takes. Where C_M^* is changing quickly the frozen-mobile approximation is therefore doing something the 0-D never does, and at 0.1 dpa it is changing quickly. The concern this raises is direct: is a march seeded at 0.1 dpa — inside the transient — carrying an error that a march seeded at 1 dpa would not?

The test is to run both. Each seed dose is run through *both* routes, on one shared snapshot grid so that no comparison is ever made between two different doses:

Case	Seed	Snapshots
A	1.0 dpa	1, 6, 11, 16, 21 dpa
B	0.1 dpa	0.1, 1, 6, 11, 16, 21 dpa— case A’s grid, plus one short interval covering the transient itself

The correction the fast step makes is the same at both seeds. Solving once for C_M^* against the seed’s own immobile state, before any immobile marching, measures how far the seeded field is from quasi-steady:

Species	seed 0.1 dpa	seed 1.0 dpa
C_v	$\times 0.643$	$\times 0.645$
C_i	$\times 0.730$	$\times 0.730$
C_{2i}	$\times 0.288$	$\times 0.263$
C_{3i}	$\times 0.280$	$\times 0.256$

Moving the seed from inside the transient to outside it changes the required correction by two parts in a thousand in C_v and not at all in C_i . The reason a 0-D seed is not quasi-steady is therefore *not* that it was taken mid-transient. It is that a 0-D seed is spatially uniform and carries no boundary layer, while the true $C_M^*(x)$ is pinned to thermal equilibrium on every Dirichlet face — a defect the fast step repairs in one solve regardless of the dose at which the seed was drawn. The spatial contrast of the relaxed field is 5.9×10^8 in C_v at both seeds.

Both seeds reach the same interior state. Case B marched from 0.1 dpa through the transient to 1 dpa — the dose at which case A was *handed* the 0-D state. Comparing what B computed there with what A was given, and then with A’s own trajectory over the following 20 dpa, all as interior means relative to the 0-D at the same dose:

Ratio to 0-D	B at 1 dpa	A at 6	A at 11	A at 16	A at 21
N_a	0.744	0.743	0.742	0.741	0.741
N_c	1.001	1.001	1.001	1.001	1.001
c_a	0.834	0.832	0.831	0.831	0.831
c_c	0.908	0.909	0.908	0.908	0.908

The columns agree to three significant figures. The 3-D interior has its own fixed point — roughly $0.74\times$ the 0-D in $\langle a \rangle$ number density, $0.83\times$ in $\langle a \rangle$ content, and indistinguishable from the 0-D in $\langle c \rangle$ number — and both seeds land on it. Case B reaches it within its first 0.9 dpa interval; case A reaches it within its first 5 dpa interval and then holds it, unchanged in the fourth digit, for the next 15 dpa.

Directly, at every shared dose. With both marches complete the comparison can be made without the intermediary of the 0-D. The ratio of the 0.1 dpa-seeded march to the 1 dpa-seeded one, interior means, at every dose the two share:

Dose [dpa]	N_a	N_c	c_a	c_c
6	1.000	1.000	1.000	1.000
11	1.000	1.000	1.000	1.000
16	1.000	1.000	1.000	1.000
21	1.000	1.000	1.000	1.000

Every entry is unity to four digits; the largest deviation anywhere in the set is 5×10^{-4} . Two marches that differ by a factor of ten in seed dose, one of them started deep inside the mobile transient, are indistinguishable from 6 dpa onward.

One row that is not a disagreement. At 1 dpa the cross-seed ratio is 1.344 in N_a rather than 1.000, and this is a comparison artifact rather than a result: case A’s “value at 1 dpa” is its 0-D seed, which has not been marched at all, while case B’s is a state marched through 0.9 dpa of frozen-mobile substeps. The ratio 1.344 is simply $1/0.744$ — the same interior-to-0-D offset seen everywhere else, read backwards. The first dose at which both cases have actually marched is 6 dpa.

The two routes relax at different rates. Case B’s IMEX at 1.1 dpa has $N_a/0\text{-D} = 0.916$, still well short of the 0.781 it settles at by 6.1 dpa; the Adaptive march is already at 0.744 by 1 dpa. The IMEX has taken a single 1 dpa step from its seed at that point, where the Adaptive march has taken twenty substeps with four fast solves. The frozen-mobile march therefore reaches the fixed point *faster* in dose than MoDELlib3’s own scheme does — the opposite of what one would expect if the splitting cadence were the accuracy-limiting factor.

The IMEX forgets its seed too. The same test on MoDELlib3’s own route — two DDomp runs differing only in seed dose, 3109 s and 3239 s, every mobile solve converged in both — gives the ratio of the 0.1 dpa-seeded run to the 1 dpa-seeded one, as interior means:

Dose [dpa]	N_a	N_c	c_a	c_c
6	0.9994	1.0000	0.9997	0.9997
11	0.9996	1.0000	0.9999	0.9997
16	0.9997	1.0000	0.9999	0.9998
21	0.9998	1.0000	1.0000	0.9998

The 0.1 dpa-seeded run reports at 6.1, 11.1, 16.1 and 21.1 dpa — integer dose steps from a 0.1 dpa seed cannot land on 6.0 — and part of the residual is that 0.1 dpa offset rather than seed memory. Whatever remains is at most 0.06% at 6 dpa and falls monotonically to 0.02% by 21 dpa. The two routes therefore agree not only on the answer but on the *insensitivity* to where the seed was taken, which is a stronger statement than either run makes alone.

Conclusion. The march forgets its seed. Seeding inside the mobile transient costs nothing that seeding outside it avoids, because the fast step re-establishes $C_M^*(x)$ from scratch at every refresh and the immobile population relaxes onto the same attractor either way. What the march is sensitive to is the *spatial* deficiency of a 0-D seed, not its dose — and that is repaired by the first fast solve. The corollary is that the offset from the 0-D visible in the table above is not a seeding artifact and not an accumulating splitting error: it is a genuine difference between the spatially-resolved interior and a well-mixed 0-D at the same dose, established early and then held.

10.7 Removing the 0-D seed entirely

If the march forgets its seed, the natural next question is whether the seed is needed at all: can the seed dose be pushed down until the 3-D solve resolves the nucleation transient itself, starting from unirradiated material? It can, and the reason is a time-scale separation that the previous sections did not need to name.

The two transients are $100 \times$ apart in dose. Taking the adaptive 0-D as the reference, the dose at which each quantity first reaches 99% of its 1 dpa value:

Mobile	99% at [dpa]	Immobile	99% at [dpa]
C_i	6.0×10^{-10}	N_c	5.9×10^{-3}
C_{3i}	1.6×10^{-8}	N_a	7.7×10^{-3}
C_v	4.2×10^{-5}	c_a	2.3×10^{-2}
		c_c	8.2×10^{-1}

The mobile species are settled by 10^{-4} dpa; loop nucleation runs from 10^{-3} to 10^{-2} dpa. That gap is precisely the assumption the operator split rests on, so *decreasing* the seed dose moves into the regime where the split is most valid, not out of it.

This also corrects a natural misreading of the fast step. It has no time derivative — `solveMobileClusters` solves $\nabla \cdot (D \nabla C) + G - R(C) = 0$ — so there is no mobile transient in either route to resolve. Both already assume instantaneous quasi-steady state. What the 3-D solve would be asked to resolve is the *nucleation* transient with the mobile field slaved throughout, which is well posed.

Only the immobile half of the seed was ever used. `run_coupled` overwrites $Y[:, 0:4]$ with the first fast solve before any immobile marching, so the 0-D seed’s mobile field never reaches the answer. Removing the seed therefore means starting the immobile population at the concentration floor — which is what both codes already mean by “no loops”, since each floors its populations and the mean loop size $r = l_a \sqrt{c/N}$ is 0/0 at exactly zero.

The loop-free limit is not singular. Running the same fast solve against three immobile states:

Immobile state	N_a	wall [s]	converged	C_v mean	C_{3i} mean
pristine	2.0×10^{-20}	211	yes	3.602×10^{-6}	8.08×10^{-12}
0.01 dpa	9.7×10^{-8}	196	yes	3.479×10^{-6}	5.43×10^{-12}
1 dpa	8.1×10^{-8}	197	yes	3.233×10^{-6}	4.08×10^{-12}

Deleting every loop raises C_v by 11% and the di- and tri-interstitial clusters by a factor of two, and costs the solver nothing: the dislocation network, not the loop population, carries most of the sink strength at this dose rate.

A march from pristine material reproduces the seeded ones. Run on a logarithmic grid $0 \rightarrow 10^{-3} \rightarrow 10^{-2} \rightarrow 0.1 \rightarrow 1 \rightarrow 6$ dpa, 20 substeps per interval, a fast solve every fifth: 4818 s, 20 fast solves, none capped. Against the adaptive 0-D through the transient, and against the seeded marches afterwards, all as interior means:

Ratio of pristine march to	N_a	N_c	c_a	c_c
0-D at 10^{-3} dpa	0.961	1.001	1.009	0.996
0-D at 10^{-2} dpa	0.871	1.000	1.148	0.892
march seeded at 0.1 dpa, at 1 dpa	1.0006	1.0000	1.0000	0.9996
march seeded at 1 dpa, at 6 dpa	0.9995	1.0000	0.9998	0.9997

Through the nucleation transient the 3-D interior tracks the adaptive 0-D to within 15%, drifting toward the $0.74\times$ interior fixed point as it should. From 1 dpa onward a march that never saw a 0-D state is indistinguishable from one that was handed it, to five parts in ten thousand. **The 0-D seed is not required.**

A by-product that pins down the boundary artifact. The whole-domain N_a at 6 dpa is 1.1484×10^{-5} from pristine against 9.5996×10^{-6} from the 1 dpa seed, a ratio of 1.196. The two runs differ only in how much dose the surface accumulated — 6 dpa against 5 — and $6/5 = 1.200$. The surface population is therefore exactly linear accumulation of cascade nucleation with no removal, to three figures, and the whole-domain mean depends on the seeding history purely through that. The interior does not: it agrees to 0.05%.

What it costs, and what it does not buy. The fast solve is steady, so refining the dose grid buys more solves at an unchanged price per solve, and the immobile side is adaptive and absorbs the stiffness for free. Two qualifications:

- **The IMEX cannot follow cheaply.** MoDELib3 steps at a fixed dtMax of 1 dpa. Resolving 10^{-3} – 10^{-2} dpa needs either a global hundred-fold cut — 2000 steps, some 86 hours at the measured 155 s/step — or a variable schedule it does not currently have. The Adaptive route gets the logarithmic grid for nothing; the IMEX does not.
- **Below $\sim 10^{-4}$ dpa the QSSA genuinely fails,** because the mobile field is still rising from zero and a steady solve is the wrong answer. Nothing has nucleated there — N_a at 10^{-5} dpa is 0.25% of its final value — so the error is confined to a window with nothing in it.

Seeding from the 0-D remains convenient when the interest is at doses well past the transient, since it skips three decades of dose for four fast solves. But it is a convenience, not a requirement, and it is the less defensible choice whenever the transient itself matters: a 0-D seed imposes a spatially uniform loop population the 3-D would never produce, including in the boundary shell where the 3-D’s own answer is qualitatively different — at 10^{-3} dpa the surface is *depleted* relative to the interior (8.69×10^{-9} against 1.16×10^{-8}), because low C_{3i} there suppresses homogeneous nucleation, and only later does cascade accumulation invert it.

10.8 What this does and does not isolate

With the fast step closed, both routes now solve the same fast problem with the same code, each against its own immobile state. Two differences remain, and neither is the integrator alone:

1. **Different immobile equations.** The Adaptive route integrates ZrMicro’s immobile right-hand side; the IMEX integrates MoDELib3’s. These are two models, not two integrations of one model.

2. **Different integrators**, as characterized in Section 7, and a different splitting cadence: FEM_EVERY substeps between fast solves against the IMEX’s one per internal substep.

Attributing any number below to the integrator alone would therefore still be wrong. Section 10.11 separates the contributions.

10.9 The two routes, compared

Both routes were run from the same calibrated seed at 1 dpa to 21 dpa. The IMEX completed 20 dose steps in 3109 s with every mobile solve converged; the Adaptive march completed the same span in 3964 s with 16 fast solves, all converged, none capped. Interior means, and the ratios to the 0-D at the same dose:

Dose	Quantity	0-D	Adaptive	IMEX	cpl/0-D	std/0-D	cpl/std
6	N_a	8.051×10^{-8}	5.983×10^{-8}	6.307×10^{-8}	0.743	0.783	0.949
11	N_a	8.051×10^{-8}	5.974×10^{-8}	6.296×10^{-8}	0.742	0.782	0.949
16	N_a	8.051×10^{-8}	5.970×10^{-8}	6.290×10^{-8}	0.741	0.781	0.949
21	N_a	8.051×10^{-8}	5.967×10^{-8}	6.287×10^{-8}	0.741	0.781	0.949
6	N_c	1.780×10^{-8}	1.781×10^{-8}	1.778×10^{-8}	1.001	0.999	1.002
21	N_c	1.780×10^{-8}	1.781×10^{-8}	1.778×10^{-8}	1.001	0.999	1.002
6	c_a	1.338×10^{-4}	1.113×10^{-4}	1.141×10^{-4}	0.832	0.853	0.975
21	c_a	1.338×10^{-4}	1.112×10^{-4}	1.140×10^{-4}	0.831	0.852	0.975
6	c_c	3.077×10^{-3}	2.796×10^{-3}	2.072×10^{-3}	0.909	0.673	1.350
21	c_c	3.077×10^{-3}	2.792×10^{-3}	2.068×10^{-3}	0.908	0.672	1.350

The route-to-route offset is constant in dose. The cpl/std column does not move in the third digit between 6 and 21 dpa for any of the four quantities. That is the diagnostic that matters. An error accumulating from the operator split, from the frozen-mobile interval, or from the first-order nodal update would compound with dose; a fixed multiplicative offset that survives 15 dpa of marching unchanged is a difference between the two immobile *models*, not between the two integrations of them. This is consistent with the non-correspondences listed in the project documentation and inconsistent with a numerical defect in either scheme.

Where they agree and where they do not. N_c agrees to 0.2%, c_a to 2.5% and N_a to 5%. The outlier is c_c at 35%, and it falls exactly where the known model differences are: the bi-pyramid vacancy family exists only in the 3-D model, and size-dependent vacancy-loop thermal emission is 3-D only where the 0-D uses a constant surrogate. Both are $\langle c \rangle$ vacancy physics, and both would act to lower the IMEX’s vacancy-loop content relative to ZrMicro’s — which is the observed direction, std/0-D = 0.672 against the Adaptive route’s 0.908. The discrepancy is not a units error: the IMEX figures pass through `modelib_immobile_to_0d` while the Adaptive figures are read directly in 0-D variables, and N_c , which shares that conversion factor with c_c , agrees to 0.2%.

10.10 Speed

The Adaptive march is $5.7\times$ faster than the IMEX solve at equal dose span. That is not a like-for-like statement about integrators: the IMEX also solves the mobile FEM boundary-value problem at every step, which the Adaptive route reads from a file rather than computing. It is a statement about the two routes as a user runs them.

Table 22: Wall clock for the full 0.1–20.1 dpa schedule, 24 cores.

Route	Integrator	wall [s]	per dpa [s]
IMEX	MoDELib3 semi-implicit, 20 substeps/dpa, nodal	2409	120.5
Adaptive ($\Delta\gamma = 5$ dpa)	CVODE BDF, <code>acc_mode=2</code> , AD Jacobian	424	21.2
Adaptive ($\Delta\gamma = 1$ dpa)	as above, mobile refreshed $5\times$ more often	526	26.3

10.11 Attribution: model, splitting, or integrator?

The decisive measurement is to place both routes against the plain 0-D model integrated over the same dose range.

Table 23: Domain-mean $\langle a \rangle$ loop number density (atom fraction). “0-D alone” is ZrMicro integrated to the same dose with no spatial coupling at all.

Dose [dpa]	0-D alone	Adaptive	IMEX	Adaptive / 0-D	IMEX / 0-D
5.1	1.03×10^{-2}	2.61×10^{-3}	1.43×10^{-4}	0.255	0.0140
10.1	1.84×10^{-2}	8.79×10^{-3}	1.53×10^{-4}	0.478	0.0083
15.1	2.14×10^{-2}	1.56×10^{-2}	1.62×10^{-4}	0.728	0.0076
20.1	2.32×10^{-2}	2.23×10^{-2}	1.72×10^{-4}	0.961	0.0074

The Adaptive route **converges onto the plain 0-D answer** as dose accumulates ($0.255 \rightarrow 0.961$): the spatially resolved mobile field delays the 0-D result but does not change where it goes. The IMEX sits about $130\times$ below the 0-D throughout.

Refining the splitting interval settles the remaining ambiguity. Halving is not enough to be conclusive, so the interval was cut by five, holding the total substep count fixed:

	$\Delta\gamma = 5$ dpa	$\Delta\gamma = 1$ dpa	change
Adaptive $\langle a \rangle$ at 20.1 dpa	2.23×10^{-2}	2.77×10^{-2}	+24%
ratio to IMEX	130	161	—

A five-fold refinement of the splitting moves the answer by 24%, and moves it *away* from the IMEX rather than toward it. So:

- the **model difference is** $\sim 130\times$ and dominates everything else by two orders of magnitude;
- the **splitting error is** $\sim 20\text{--}25\%$ at $\Delta\gamma = 5$ dpa, and is not what separates the routes;
- the **integrator difference is bounded well below both**, consistent with Section 7.

The $\langle c \rangle$ family shows the mirror image: the IMEX holds 1.78×10^{-8} while the Adaptive route sits at 5.50×10^{-10} , a factor 32 the other way, flat in dose in both routes.

10.12 A discrepancy with the calibration record

This conflicts with what `ZR3D_GHONIEM_CHANGES.md` records. Its issue 14 states that after moving coalescence into the implicit update, “ $\langle a \rangle$ loop density went from $3\text{--}4\times$ the 0-D to within $\sim 30\%$ ”. Here MoDELib3 sits at $0.7\text{--}1.4\%$ of the 0-D — roughly $100\times$ *below*, not within 30%.

One difference from the calibration conditions is visible in the run itself and should be checked first. The calibration used the tutorial’s *generated* microstructure; this run seeds MoDELib with the 0-D state at 0.1 dpa, for which $N_a = 4.94 \times 10^{-4}$. MoDELib immediately coalesces that down to 1.43×10^{-4} by 5.1 dpa and then holds it essentially flat (1.72×10^{-4} at 20.1 dpa) while the 0-D

grows a hundredfold. Whether that is the seeded start putting MoDELib outside its calibrated regime, or a genuine regression in the calibration, is not resolved here. It is flagged rather than explained, and no physics conclusion should be drawn from these fields until it is.

10.13 Output

The run writes the reference layout under `output/<stamp>_<hash>_coupled_vs_standalone/: 0d/, 3d/standalone/ and 3d/coupled/, gb/` for both routes, `tables/` with the comparison in Markdown, CSV and JSON, and `ev1_coupled/` holding the marched CD fields as complete, restartable MoDELib configurations. `run_coupled_vs_standalone.py` drives it, and Section 14 of `coupled_0d_3d_ZrMicro.ipynb` calls that driver.

11 Why the slow step is tractable

A 500 nm hexagonal case advances 90 617 quadrature points, each carrying $N_{\text{eq}} - N_{\text{mob}} = 15$ integrated equations, over every substep. Read as a total that is $\sim 1.09 \times 10^6$ scalar ODEs per substep and $\sim 2.6 \times 10^7$ over a 24-substep march. Stated that way it invites the obvious objection: a tightly coupled stiff system of that size is not integrable in any reasonable time, and experience with tightly coupled reaction–diffusion systems says so plainly.

The objection is correct, and it does not apply, because this is not one system of 1.09×10^6 equations. It is 72 494 *independent* systems of 15 equations each (after deduplication, Section 9.3). This section sets out why the decoupling is exact rather than an approximation adopted for speed, what it is worth arithmetically, and — the part most easily lost — where the tight coupling actually went, since it certainly did not disappear.

11.1 The decoupling is a property of the split, not of the solver

`RateEquations.ode_system(t,y)` takes a single point’s state vector. No neighbouring state appears in it anywhere; there is no stencil, no gradient, no exchange term. The batch driver (`coupling.immobile.run_immobile_step`) therefore dispatches each point as an independent case of the OpenMP `-batch_file` mode, dynamically scheduled across the available cores with one reusable CVODE workspace per thread (Section 3.3).

That is legitimate for one reason only: *the sole mechanism coupling neighbouring points is diffusion of the mobile species*, and the slow step freezes them. With `freeze_mobile=1` the four mobile concentrations are held at the $C_M^*(x)$ delivered by the preceding fast solve, and a point’s immobile equations then depend on nothing but its own loop populations, its own ρ_N , and those four pinned values. The spatial operator is absent from the slow step by construction.

So the decoupling is earned by the operator split. The batch mode does not create it; it harvests it. Any change that let the immobile equations see a neighbour — a gradient term, a non-local sink — would destroy the property and with it the entire cost argument below.

11.2 What the decoupling is worth

An implicit integrator applied to a coupled system of dimension N pays an $N \times N$ factorization. For a dense factorization the cost is $\mathcal{O}(N^3)$, and the two arrangements differ by ten orders of magnitude:

Sparsity softens the coupled entry without rescuing it. A three-dimensional finite-element Jacobian factorizes at roughly $\mathcal{O}(N^2)$ with fill growing as $\mathcal{O}(N^{4/3})$, so the coupled arrangement remains unreachable in both time and memory — and it would be paid at every Newton iteration of every time step, not once.

Stiffness localizes in the same way, and this is worth stating separately because it is the point at which intuition from tightly coupled systems misleads. Each point’s system *is* genuinely stiff: the reaction rates span decades, and CVODE takes ~ 100 internal steps per point per substep,

Table 24: Cost of one implicit factorization, 500 nm case ($q = 90\,617$ points, dedup $\times 1.25$, 15 equations per point).

Arrangement	Factorization	flops
Split (this scheme)	$72\,494 \times 15^3$	2.4×10^8
Fully coupled	$(1\,359\,255)^3$	2.5×10^{18}
Ratio		1.0×10^{10}

which is why BDF is used rather than an explicit method. But the cost of stiffness scales with the dimension of the *coupled block*, not with how many independent blocks exist. A 15×15 dense LU is ~ 1100 flops. Paying that 72 494 times is free; paying it once at $N = 1.36 \times 10^6$ is not.

11.3 Where the tight coupling went

The coupling has not been removed from the physics. It has been moved to the one place where its cost is bearable.

The tightly coupled problem is the *fast* step: a nonlinear algebraic system in $N_{\text{mob}} \times q = 362\,468$ unknowns, solved by Newton iteration with a sparse direct factorization (`Eigen::SimplicialLLT`). It is expensive exactly as the objection predicts — 925 s measured on the 500 nm case, against 28.3 s for a complete slow sweep over all 90 617 points. Tight coupling costs what tight coupling costs.

What the split buys is not a cheaper coupled solve but a *rarer* one. `solveMobileClusters` has no time derivative: it returns the steady mobile field for the immobile state currently held. A steady problem need not be re-solved as time advances — only when the state it was solved against has moved enough to matter. The coupled problem is therefore solved 8 times over a march instead of at every internal time step.

The counterfactual follows from the same measurements. At 925 s for 5 Newton iterations, one linear solve costs ~ 185 s. Integrating the mobile block implicitly in time, with a refactorization every fifth step, would need

$$\frac{100 \text{ steps} \times 24 \text{ substeps}}{5} \approx 480 \text{ factorizations}, \quad 480 \times 185 \text{ s} \approx 25 \text{ h},$$

and that is the four-species mobile block alone, not the full nineteen-variable system. The measured slow side of the same march is 11.3 minutes.

11.4 The price, and the check

The bill arrives as splitting error, first order in the interval between fast solves — `fem_every` substeps, not the dose-snapshot spacing (Section 10). That is why `fem_every` is the accuracy-versus-wall-clock dial of the whole scheme, and why a value which does not divide `substeps_per_interval` is a defect rather than a preference: the cadence counts substeps over the whole march, so a non-dividing value drifts and can leave entire dose intervals with no fast solve at all, freezing the mobile field across a full interval.

The structure makes a falsifiable prediction: with the points independent, the slow step must scale *linearly* in their number. It does.

Table 25: Slow-step cost against problem size, measured.

Case	CD points	per substep [s]	per point [s]
200 nm prism	27 720	9.3	3.35×10^{-4}
500 nm prism	90 617	28.3	3.12×10^{-4}

A $3.27\times$ increase in points costs $3.04\times$ the time. Superlinear growth in this table would be the signature of neighbour coupling having leaked back into the slow step, and would show up in the wall clock before it showed up in the physics.

12 Construction of Discrete Loops

The cluster-dynamics state carries, at every node and for every immobile family k , exactly two numbers: a loop number density n_k and a stored defect content c_k . Dislocation dynamics needs something else entirely — a finite list of loops, each with a position, a radius, a habit-plane normal and a Burgers vector. This section sets out the conversion, why it differs from the one in `Modelib-fullCD`, and how its output enters a DD simulation.

Implementation: `disloccluster_code/post/discrete_loops.py`.

12.1 What the earlier figures were, and were not

The `loops_*` overlays produced by `modelib_fields._overlay_family` are a *size map drawn as an ensemble*, not a microstructure. Platelet centres are CD node coordinates; the platelet *count* is whatever a greedy non-overlap packing admits, capped at 2500; and the drawn radii carry a fixed exaggeration ($\times 7$ for $\langle a \rangle$, $\times 1.5$ for $\langle c \rangle$). The gap between that and a microstructure is not subtle: at 10^{-4} dpa the figure draws 2500 platelets into a domain that physically contains $2.7 \langle a \rangle_1$ loops. Nothing in it can be handed to DD.

12.2 Mathematical method

Count. The number of loops of family k in a region R is not a parameter. It is

$$N_k = \sum_{j \in R} n_k(\mathbf{x}_j) V_j, \quad (4)$$

where V_j is the volume attached to CD node j . Because `cdNodes.txt` carries positions only and no connectivity, V_j is obtained by Monte-Carlo Voronoi assignment — draw points uniformly in the domain, assign each to its nearest node, take the counts as volume shares — the same weights the volume averaging uses. Loop positions are then an inhomogeneous Poisson sample of intensity $n_k(\mathbf{x})$, so spatial structure survives into the discrete population rather than being averaged away.

Size. The defects per loop follow from dividing content by density, and the radius from storing them in a disc one Burgers vector thick:

$$m(\mathbf{x}) = \frac{c_k(\mathbf{x})}{n_k(\mathbf{x}) \Omega / b^3}, \quad \pi r^2 |\mathbf{b}_k| = m \Omega \implies r(\mathbf{x}) = \sqrt{\frac{m(\mathbf{x}) \Omega}{\pi |\mathbf{b}_k|}}. \quad (5)$$

This is defect-count conservation, and it is the same rule `fullCD` applies in `initializeDiscreteClimbLoops`.

Crystallography. These are pure climb loops, so the habit-plane normal is parallel to the Burgers vector. Table 26 gives the four families.

The three $\langle a \rangle$ entries are the members of $\frac{1}{3}\langle 11\bar{2}0 \rangle$. The material file writes them in the three-index lattice basis as $(1, 0, 0)$, $(0, 1, 0)$ and $(-1, 1, 0)$, which map to the Miller–Bravais indices above and to Cartesian $(1, 0, 0)$, $(0.5, 0.866, 0)$ and $(-0.5, 0.866, 0)$ — all of unit length, hence crystallographically equivalent at zero applied stress, as they must be.

Hexagons for $\langle c \rangle$ are not a drawing preference: basal vacancy loops in Zr facet on $\langle 10\bar{1}0 \rangle$ -type edges. $\langle a \rangle$ loops are round enough that a 16-gon is indistinguishable from a circle at these radii.

Table 26: Discrete loop families. `planeID` indexes `singleCrystal->slipSystems()`; with `enabledSlipSystems=fullBasal fullPrismatic` that list is six basal systems (0–5), then the prismatic planes in pairs.

Family	Habit plane	Burgers vector	$ \mathbf{b} $	Sides	planeID	Type
$\langle c \rangle$	basal (0001)	$\frac{1}{2}[0001]$	$0.8165b$	6	0	vacancy
$\langle a \rangle_1$	prismatic	$\frac{1}{3}[2\bar{1}\bar{1}0]$	$1.0b$	16	6	interstitial
$\langle a \rangle_2$	prismatic	$\frac{1}{3}[11\bar{2}0]$	$1.0b$	16	8	interstitial
$\langle a \rangle_3$	prismatic	$\frac{1}{3}[\bar{1}2\bar{1}0]$	$1.0b$	16	10	interstitial

The polygon carries the area, not the radius. A regular n -gon of circumradius R has area $\frac{n}{2}R^2 \sin(2\pi/n)$, which for a hexagon is only 82.7% of πR^2 . Since the conserved quantity is the number of point defects stored, it is the *area* that must match, so the exported circumradius is

$$R = r \sqrt{\frac{2\pi}{n \sin(2\pi/n)}}, \quad (6)$$

which is 1.0996 for the hexagon and 1.013 for the 16-gon. `fullCD` does not apply this correction, so its 12-gons under-fill by 2.3%.

12.3 Coalescence, and why it must be bounded

Where loops are dense enough to overlap, a second pass merges them area-conservingly,

$$r_{\text{new}} = \sqrt{\sum_i r_i^2}, \quad (7)$$

placing the merged loop at the area-weighted centroid so the first moment of the stored content is preserved as well.

Iterating this to a fixed point does not work, and the reason is structural rather than numerical. Holding the total area $N\pi r^2$ fixed while the mean spacing is $d = (V/N)^{1/3}$,

$$r \sim N^{-1/2}, \quad d \sim N^{-1/3} \quad \implies \quad \frac{2r}{d} \sim N^{-1/6}. \quad (8)$$

Every merge lowers N , so **every merge raises $2r/d$** : area-conserving coalescence cannot repair overlap, it can only worsen it. Run unbounded on the $\langle c \rangle$ population at 1 dpa and 186 loops fuse into a single loop of radius 650 nm, larger than the 500 nm box. Two bounds keep it physical.

- **Coplanarity.** Two loops may merge only if their centres lie within one interplanar spacing of a common habit plane — the statement that loops on different parallel planes cannot become one loop without climbing. `fullCD` does not test this; it groups by finite-element adjacency.
- **Spacing guard.** A pass whose result would push $2\langle r \rangle/d$ above unity is rejected and the population is reported *saturated*. Saturation is not a failure: it says the continuum state admits no non-overlapping discrete representation, which is precisely the regime the model’s own coalescence channel exists to describe.

In the 500 nm interior population $\langle c \rangle$ crosses $2r/d = 1$ at 0.546 dpa and is flagged saturated for the remaining 26 of 100 dose steps; $\langle a \rangle$ never exceeds 0.11.

12.4 Differences from MoDELlib-fullCD

`fullCD` performs the conversion once, at `clusterDiscretizationTime`, in `ClusterDynamics<dim>::initializeDiscr`. It ranks finite elements by defect content, greedily absorbs vertex-neighbours until a group holds `clusterDiscretizationFactor` loops, and emits *one* 12-sided loop carrying the group’s entire content, centred on a random tetrahedron barycentre and snapped to the nearest crystallographic plane. Table 27 sets the two side by side.

Table 27: Construction of discrete loops: MoDELib-fullCD against the present scheme.

	MoDELib-fullCD	Present scheme
Loop count	N_{phys}/F , set by the user parameter <code>clusterDiscretizationFactor = F</code>	$N_k = \sum_j n_k(\mathbf{x}_j) V_j$, the physical count
Grouping	Greedy FE vertex-neighbour growth, applied everywhere	None; coarsening only where geometry forces it
Size resolution	One radius per element group	One radius per loop, from the local c_k/n_k
Placement	Random tetrahedron barycentre of the group	Inhomogeneous Poisson sample of $n_k(\mathbf{x})$
Polygon	12 sides for every family	6 ($\langle c \rangle$, faceted) / 16 ($\langle a \rangle$)
Polygon area	Circumradius = r ; under-fills by 2.3%	Area-matched, Eq. (6); exact
Coplanarity test	None	Within one interplanar spacing
Overlap handling	Implicit, uniform, up front	Explicit, local, with a saturation guard
Timing	One shot at a prescribed time	Any dose step, from the stored march state
Cost	Sort and regroup N_{ele} elements per species, with repeated FE evaluation	$O(N_{\text{node}})$ for the count, then $O(N_{\text{loop}} \log N_{\text{loop}})$

How much more precise. The coarsening is not free. Under the dispersed-barrier model the hardening increment scales as $\Delta\tau \propto \sqrt{N_{\text{loop}} d_{\text{loop}}}$ with $d_{\text{loop}} = 2r$. Replacing F loops of radius r by one of radius $r\sqrt{F}$ conserves area but gives

$$\frac{\Delta\tau_{\text{coarse}}}{\Delta\tau_{\text{exact}}} = \sqrt{\frac{(N/F)(2r\sqrt{F})}{N(2r)}} = F^{-1/4}, \quad (9)$$

so a bundling factor of $F = 10$ *understates* radiation hardening by $1.78\times$ and $F = 100$ by $3.16\times$. The present scheme carries $F = 1$ everywhere the loops fit, so this error is absent by construction.

How much more efficient. The comparison that matters is not the conversion cost — both are negligible beside a DD solve — but the number of loops the DD run must then carry. `fullCD` reduces that count by coarse-graining, at the cost of Eq. (9). The present scheme reduces it instead by *selecting the physically meaningful region*: the whole 500 nm domain at 10 dpa holds 56,290 loops, but 98.6% of them sit in the near-boundary shell, where cascade nucleation is uniform while the only removal channel — coalescence driven by the absorbed mobile flux — vanishes against the Dirichlet faces. The interior population is 801 loops. That is a $70\times$ reduction *with no coarse-graining and no precision loss at all*, against the $F^{1/4}$ hardening error that a $70\times$ bundling factor would have incurred, namely $2.9\times$.

12.5 Interface with the dislocation dynamics input files

A MoDELib3 DD simulation reads a single `inputFiles/` directory. Reviewing `tutorials/periodic_density/gener` together with the `clean_run.sh` scripts of the `zrmicro_*` cases, the required set is that of Table 28.

The microstructure file. Every field of the `aLoops` individual specification is per loop, so all four families go into one file:

```
type=aLoops; style=individual;
planeIDs=...; loopRadii_SI=...; loopSides=...; loopCenters=...; isVacancyLoop=...
```

`planeIDs` selects basal or prismatic, `loopSides` gives the hexagons and 16-gons of Table 26, and `isVacancyLoop` sets the polarity, which `aLoopGenerator::generateSingle` then reconciles against

Table 28: Input set for a DD simulation. Only the last two rows are produced by the loop construction; the rest are the standard MoDELib deck.

File	Role
DD.txt	Solver controls: <code>useDislocations=1</code> , <code>useClusterDynamics=0</code> for a pure DD run, <code>glideSolverType</code> , <code>climbSolverType</code> , <code>timeSteppingMethod</code> , <code>Nsteps</code> , <code>Lmin/Lmax</code> , <code>remeshFrequency</code> , <code>outputFrequency</code>
<Material>.txt	Zr3d_ghoniem.txt, with <code>enabledSlipSystems=fullBasal fullPrismatic</code> — which is what fixes the <code>planeID</code> values of Table 26
ElasticDeformation.txt	ExternalStress0 and ExternalStrainRate: the loading that drives plastic deformation
polycrystal.txt	Written by PolyCrystalFile: <code>mesh</code> file, <code>boxScaling</code> , <code>X0</code> , <code>periodicFaceIDs</code> , grain orientation, <code>absoluteTemperature</code>
<mesh>.msh	The domain. For the present case a 500 nm cube, 1546.55 <i>b</i> per side
initialMicrostructure.txt	One <code>microstructureFile=<name></code> ; line per microstructure to superpose
aLoops_<dose>.txt	Produced here. The loop population as an <code>aLoops/individual</code> specification
loops_<dose>.csv	Produced here. Per-loop record — family, position in <i>b</i> and nm, radius, circumradius, Burgers vector, normal, merge count — for analysis outside MoDELib

the polygon winding so that the sign of **b** matches the loop’s area normal. Both branches of that generator emit `SESSILELOOP`, which is correct: for a climb loop $\mathbf{b} \cdot \mathbf{n} \neq 0$, and MoDELib3 assigns `loopType` on exactly that test.

Two corrections applied on export. First, `loopRadii_SI` carries the *circumradius* of Eq. (6), not the disc radius, because the generator places vertices on a circle of the radius it is handed. Second, the type key is written `type=aLoops`; with the plural: the generator dispatches on `type=="aLoops"`, and the shipped example `Library/Microstructures/aLoopsIndividual.txt` writes `type=aLoop`;, which matches no branch and is therefore silently not generated. `aLoopsDensity.txt` in the same directory has it right. This is a defect in the MoDELib3 library example, not in the specification parser.

The $\langle c \rangle$ Burgers vector disagrees between the two codes. The cluster-dynamics material file gives the $\langle c \rangle$ family a *full* [0001] Burgers vector, $|\mathbf{b}| = c/a = 1.633b$, while `aLoopGenerator`’s basal branch builds a *half* [0001] loop, $|\mathbf{b}| = 0.8165b$ — the physically standard basal vacancy loop in Zr. The invariant handed from CD to DD is the number of vacancies stored, not the radius, so Eq. (5) is evaluated with the *DD* Burgers magnitude. The consequence is that $\langle c \rangle$ radii come out a factor $\sqrt{2}$ larger than the CD-internal value, 47.7 nm rather than 33.7 nm at 10 dpa. This is a genuine model inconsistency and should be resolved on the cluster-dynamics side.

Workflow. Copy `aLoops_<dose>.txt` into `inputFiles/`, name it from `initialMicrostructure.txt`, then

```
microstructureGenerator . → evl/evl_0.txt → DDomp .
```

The generator builds the loop network — nodes, loop links, glide planes — from the specification and writes the initial configuration; `DDomp` then runs the plasticity. Because a separate file is written at every dose step, a deformation simulation can be started from the irradiated state at any dose on the march without re-running either the cluster dynamics or the coupling.

12.6 Results

Table 29 gives the interior population of the 500 nm pristine-start Adaptive run. The count rises to a maximum of 1024 loops near 0.027 dpa and then *falls* to a plateau of about 800: that maximum is the crossover from nucleation-dominated to growth-dominated evolution, and it is visible only because the dose grid is resolved logarithmically over five decades.

Table 29: Interior discrete loop population, 500 nm cube. d is the mean inter-loop spacing $(V/N)^{1/3}$.

Dose [dpa]	$N_{(c)}$	$r_{(c)}$ [nm]	$2r/d$	$N_{(a)}$ each	$r_{(a)}$ [nm]	Total
10^{-4}	13	2.41	0.023	4	1.53	25
0.0105	187	3.45	0.079	283	3.18	1036
0.107	187	15.6	0.356	233	4.44	886
0.977	186	47.4	1.081	205	4.70	801
10	186	47.7	1.089	205	4.69	801

As a check on plausibility, the interior $\langle a \rangle$ density at 10 dpa is $4.9 \times 10^{21} \text{ m}^{-3}$, squarely within the range reported for irradiated Zr at that dose.

Files. `dislocluster_code/post/discrete_loops.py` (new), `dislocluster_code/post/volume_average.py` (Voronoi weights), `dislocluster_code/post/gb.py` (face distances), `Docs/DisloCluster Manual/continuum_to_discrete.py`. Output per run directory: `discrete_loops/` with `aLoops_<dose>.txt`, `loops_<dose>.csv`, `loops_<dose>.png` and `loops_<family>_<dose>.png` for every dose step, and `manifest.json` carrying counts, radii, spacings, the $2r/d$ ratio before and after coalescence, and the area-conservation and saturation flags.

12.7 From an offline export to a runtime handoff

Everything above converts the continuum loop field into a discrete population *for export and visualization*: it does not feed back into a solve. This section records the machinery that closes that loop, added on 16 August 2026. The physics of the conversion is unchanged — the radius rule, the coalescence bound and the crystal-aware placement are all as described above — and what is new is when it happens, what is removed from the continuum in exchange, and what makes the resulting climb solve affordable.

When. `post/coarsening.py` forms the Avrami overlap fractions per interior node per family and reports d_{coarsen} , the dose at which they cross a threshold φ^* . It is fed inside the immobile substep loop, so the crossing is resolved to a substep rather than interpolated across whatever gap the dose list leaves — which had bracketed it only to a factor of three. Three independent 500 nm marches report $d_{\text{coarsen}} = 0.4$ dpa for the basal family and no crossing for any prismatic family, identically.

What is exchanged, and the ledger that proves it. Once a loop is discrete it is a sink through the Green’s function; if it is *also* still in the continuum immobile field it is a sink through `ImmobileSinks` as well, and it absorbs twice. The same defect has a second face: `DislocationQuadraturePoint::cCD` samples the continuum mobile field at the loop line, so such a loop sees its own depletion twice over. Removing the transferred share from the continuum fixes both at once.

Three invariants are audited across every transfer — loop number, stored defects, and sink strength. The first two are conservation statements; the third is a modelling statement, and it is the one that catches mistakes. The ledger found three separate leaks that inspection had not:

1. `microstructureGenerator` silently refuses any loop whose nodes fall outside the grain. Discovering that *after* removing the family from the continuum loses those defects, so the refusal is now predicted — same convex hull, same exported polygon, same criterion — and the refused share stays in the continuum. It must be counted in *defects*, not loops: one refusal out of six was 83% by count and 98% by defects, because the refused loop happened to be the smallest. Proximity to the surface decides refusal, not size.
2. The population is drawn as $\text{round}(\sum_j n_j V_j)$ loops, an integer from a real expectation, so it misses the continuum total by the rounding: 1.6% at 98 loops and 15.4% at 6. Rescaling every radius by $\sqrt{C_{\text{target}}/C_{\text{drawn}}}$ fixes the total exactly and leaves every ratio between loops untouched.
3. Sink strength jumps at the handoff by $\sqrt{b_{cd}/b_{dd}} = 1.414$ — the continuum sizes a basal loop with the full $\langle 0001 \rangle$ Burgers vector and the discrete side with the half — times $1/\text{loopSinkScale} = 3.43$. Coalescence at transfer moves it a further $5.3\times$ on the measured case.

With the first two closed, continuum-after plus discrete equals continuum-before to 10^{-16} . The third is reported and never asserted on: it is a decision, and it must be settled before a transfer feeds a solve.

Whether a transfer is possible at all. A discrete loop must fit inside the crystal, and that is not automatic. On the 500 nm case the kept fraction is 0.997 at 0.1 dpa and 0.55 from 1 dpa onward; the plateau is purely geometric, since a loop of mean radius 48 nm in a $500 \times 433 \times 800$ nm prism must clear every wall, giving $(404/500)(337/433)(704/800) = 0.56$. On the 200 nm case the kept fraction is *exactly zero* — the basal radii reach 65 nm against a 173 nm extent, so no loop fits. The transfer declines rather than handing DD a population it would refuse, but the lesson generalizes: **a domain must be chosen against the loop size it will reach, not only against the mesh cost.**

What makes the climb solve affordable. `GalerkinClimbSolver` assembles over every ordered pair of segments, and that assembly is 100% of the climb cost — the sparse path is dead code, and the live path is lumped, so its “solve” is one scalar division per node. Reducing the number of unknowns therefore buys nothing; only reducing the number of *pairs* does.

Truncating the sum is legitimate because the kernel is not bare $1/r$, which could not be truncated — a growing loop is a net sink, its monopole does not vanish, and a shell at radius r contributes $\sim r$ — but screened as e^{-kr}/r by the same k^2 that `ImmobileSinks.h` already assembles. Independently: the continuum field already carries the mean-field response of the whole population, so the discrete sum must supply *only* the near-field correction, and extending it further would double count in exactly the way the ledger above exists to prevent.

The cutoff is the optional material key `climbNeighborCutoff_b`; zero or absent means no cutoff, so every case written before it existed is unaffected. Verified three ways on a real discrete case: with the key absent and with an enormous cutoff the results agree *bit-for-bit* in `ev1` and in every F field, which is the test of the distance logic itself, while a finite cutoff differs. The convergence sweep, on the basal plastic-distortion rate:

R_c/L_s	1	2	3	4	6
error	19.3%	9.6%	3.9%	0.25%	0.000%

so the default is 4, not the 3 first proposed — a case where the acceptance test rejected the value the same plan had recommended. The zeros at $R_c/L_s \geq 6$ are a finite-size artifact: the cutoff there exceeds the extent of the loop cloud, so nothing is excluded. Together with separating the

rendering polygon count (64 sides, chosen for figures) from the exported one (12), the assembly is $274\times$ smaller than the naive version.

Status. Each piece is validated individually. What has not yet been completed is a full march that fires the transition at d_{coarsen} and accumulates growth strain through $\mathbf{F}$ across the whole dose range; until that exists, the sink-strength discontinuity remains an open modelling decision.

13 A self-consistent slow solver

Everything up to here made the *transport* between the two codes exact: the handoff of §6 replaced four scalars with a per-node field, and the criterion of the anisotropy plan governs `solveImmobileClusters`. What remained was that the two sides still integrated *different immobile models*. This section closes that, as an option.

13.1 The inconsistency

The slow step of the coupled march is ZrMicro’s, not MoDELib3’s. It carries an aligned/non-aligned split of two loop types — iL , aiL , vL , avL — and biases its sinks with two fitted scalars δ_i, δ_v . MoDELib3 carries one basal $\langle c \rangle$ and three prismatic $\langle a \rangle$ variants, and biases its sinks with the anisotropy of the diffusion tensor. Neither structure maps onto the other, so the field bridge had to *invent* information in both directions: outward, `immobile_0d_to_modelib` split the $\langle a \rangle$ total over a_1, a_2, a_3 by a weight vector; inward, `modelib_immobile_to_0d` reconstructed the aligned/non-aligned ratio from the previous state. A round trip through the bridge was therefore not the identity, and no amount of care in the transport could make it one — the information genuinely was not there.

The second consequence is sharper, because it silently voided a result. The legacy model applies a single $Z_{i,a}$ to the whole interstitial arrival $\omega_i(C_i + 2C_{2i} + 3C_{3i})$, and it gives all three interstitial species the *monomer* jump frequency. Giving C_{2i} a different directionality from C_i therefore changes nothing in it. That is exactly the structure of the Li et al. parameter set — anisotropy on the clusters only — and it is why that set produced no co-growth here. The model had no slot for the effect.

13.2 One switch, two formulations

`SOLVER['loop_model']` selects the formulation; it is orthogonal to the Option A/Option B choice of *integrator*. Under `loop_model = 1` the eight immobile slots become

$$y_{4..11} = [n_c, n_{a1}, n_{a2}, n_{a3}, c_c, c_{a1}, c_{a2}, c_{a3}],$$

which *is* MoDELib3’s CD immobile block in MoDELib3’s order, and the capture efficiencies become Woo’s, built from the same p_m that `staging/anisotropy.py` writes the migration-energy tensor from:

$$Z_m^{\text{basal}} = Z_m^0 p_m, \quad Z_m^{\text{prismatic}} = Z_m^0 \frac{1}{2} (p_m + p_m^{-2}),$$

identical to `ClusterDynamicsParameters::loopDADbias`. Absorption of species m by family k is $\phi_{km} = S_k \bar{D}_m Z_{\text{row}(k),m} c_m |m|$, with $S_k = (\ell_k/\ell) s_k \sqrt{n_k c_k}$ built from the family’s *own* radius scale $\ell_k = \sqrt{\Omega/\pi b_k}$ and `loopSinkScale`, the same array `ImmobileSinks` multiplies by. The full equations are in the manual, §“Two formulations of the slow step”.

Architecturally the payoff is that **the bridge collapses to an identity**:

$$\text{out}[:, 0:4] = Y[:, 4:8] / \text{omega} \quad \text{out}[:, 4:8] = Y[:, 8:12]$$

and its inverse likewise. Nothing is lumped, nothing is split, and the round trip is exact — *because there is no longer any information that only one side holds*. That is a stronger statement than “the mapping was improved”.

13.3 Where the parameters come from, and why it matters

p_m , Z_m^0 and s_k are read from `Zr3d_ghoniem.txt`, not from the workbook. This is deliberate and is the whole point of the exercise: that file is the single source both sides already share, and it is the file `anisotropy.py` generates the migration energies into, from one p_m per species. Reading the efficiencies from anywhere else would recreate the exact failure this option exists to remove — the code simultaneously believing that diffusion is isotropic and that the sinks are biased by its anisotropy.

13.4 Verification: bit-identity of the fitted path

The legacy formulation is the fitted one, so the acceptance test is not “similar” but *bit-identical*. `loop_model = 0` reproduces the pre-change binary exactly, `sha256 8bcc7780...` on a 25-point integration.

That test paid for itself twice. First, an early build of the branch declared `growth_iL` and friends inside the new branch, so the legacy assembly below no longer compiled — and a `grep -c " error "` on the build output was misread as a count of something else, a stale binary was tested, and bit-identity was very nearly reported for code that had not been rebuilt. Second, once it did build, the first attempt *failed* the test: `ff2980c4` against `8bcc7780`. The cause was that the new assembly regrouped `growth + nuc + G` as `growth + (nuc + G)` — algebraically identical, and the tenth significant digit moved. Floating-point addition is not associative, and the 28-parameter fit was made against the original association.

The legacy `ydot` assembly and the paired accumulator sums are therefore kept *verbatim* inside `if (P.loop_model == 0)`, and the shared family-slot arrays are populated from them by assignment only. The self-consistent branch writes its `ydot` in a loop over slots; the legacy branch must not be rewritten that way.

The new mode’s own acceptance test is a symmetry: at equal `variant_frac` and zero resolved stress the three $\langle a \rangle$ variants must be indistinguishable, and they are — $7.5884177301 \times 10^{-7}$, ...300, ...302, agreeing to eleven digits, the residual being roundoff in CVODE’s dense factorization rather than asymmetry in the model.

13.5 What the formulation change costs, and the attribution

At 10 dpa on the calibrated 0-D set, route A anisotropy, the self-consistent model gives N_a **30.2** $\times$ the legacy value, c_c 3.36 $\times$, c_a 1.66 $\times$, and N_c essentially unchanged (0.96 $\times$).

That is a large enough departure to demand an attribution rather than a shrug, and the individual rates turn out *not* to be the cause. At the fitted parameters the two models agree almost exactly on them: $Z_i^{\text{prismatic}} = 1.072114$ against $Z_{i,a} = 1.072113$ (by construction — Z_m^0, p_m were fitted to reproduce the phenomenological Z); the $\langle a \rangle$ sink prefactor ratio $(\ell_a/\ell_c)s_a = 1.000464$; the $\langle c \rangle$ ratio $s_c/Q = 1.012493$. The $\langle a \rangle$ prefactor agreement is a coincidence worth recording: $\ell_a/\ell_c = \sqrt{b_c/b_a} = 1.2627$ and the fitted $s_a = 0.792317$ are reciprocals to 0.05%, so MoDELib3’s sink-scale calibration happens to undo the fact that the legacy model sizes its $\langle a \rangle$ sinks with the $\langle c \rangle$ Burgers vector.

The cause is **resolving the three prism variants**, and the attribution is by construction rather than by argument: concentrating 98% of the $\langle a \rangle$ nucleation back into a_1 — reassembling the lumped population and changing nothing else — brings N_a from 30.2 $\times$ to 1.52 $\times$ and c_a from 1.66 $\times$ to 1.08 $\times$. The mechanism is coalescence: like-loop coarsening removes loops at $\nu_{LL}\phi_{LL}n$ with $\phi_{LL} = 1 - \exp(-\kappa_{LL}\frac{4}{3}\pi r^3 n)$, so three families of $n/3$ coarsen much less than one family of n . They should: $\langle a \rangle$ loops of three distinct Burgers vectors on three distinct prism planes do not coalesce into one loop, and the legacy model lets them.

Two qualifications, both load-bearing. MoDELib3’s coalescence loop is itself per family (`ClusterDynamicsFEM.cpp`, `for(int k=0;k<nF;++k)` over `Nvol=n(k)`), so `loop_model = 1` agrees with

the 3-D code *by construction* — that is the stated goal, not independent evidence that per-variant coalescence is right. And the truth is between the two: cross-variant interaction is fully on in the legacy model and fully off here, while real loops of different Burgers vectors do interact, through junction reactions and through each other’s diffusion fields. The c_c column shows the coupling that does survive — the $\langle a \rangle$ change reaches the $\langle c \rangle$ family through the shared mobile pool, there being no direct term between them.

The parameter set is stale for the new mode. The 28 parameters were fitted against `loop_model = 0`. Every number in this section therefore measures a formulation change on an uncalibrated set, and none of it says which formulation matches experiment. This compounds with the Ω correction, which already invalidated the fit.

13.6 One combination is refused

`COUPLING['discrete_transition'] = True` with `loop_model = 1` raises at validation time, before anything expensive runs. `coupling/transition.py` addresses the immobile state by the legacy slot names, which under the self-consistent layout name a different family in every slot. The failure mode is the one this document keeps returning to: it would not crash, it would produce a well-formed set of discrete loops drawn from the wrong population. Porting the handoff is the natural next piece of work.

14 Summary and recommended next steps

Table 30: Summary of outcomes against intent.

Change	Target metric	Outcome
AD Jacobian	residual evaluations	−33%; wall neutral
9×9 reduction	LU flops	−92%; steps −18%; wall −4%
	step-size pathology	2048 roundoff warnings $\rightarrow$ 0
Workspace reuse	solver setup cost	march wall $\times 1.67$ at $q = 1024$
Field handoff	information exchanged	4 scalars $\rightarrow$ 289 380 values
Full 1–26 dpa march	same-work solver speed	$1.95\times$ (Table 10)

Two findings are worth carrying forward independently of the changes themselves. First, the accumulators were quietly setting the initial step of every coupling substep, and removing them from the error test cures a roundoff-limited first step that had been occurring 2048 times per march (Section 5.4). Second, the two codes integrate the same immobile equations to very different error levels — first order with no error control on the 3-D side, adaptive to 10^{-6} on the 0-D side — which is a systematic 0-D/3-D discrepancy distinct from the three physics non-correspondences on record (Section 7).

Items 1–3 of the original list have been implemented and measured (Section 9); what follows is what remains, ordered by expected return.

1. **Drive MoDELib in process.** This is now the binding constraint. Item 1 of the previous version of this list — put the FEM mobile solve inside the march loop — has been done (Section 10.1), and doing it exposed the cost: every fast solve is a separate DDomp invocation, so the mesh load, the FE assembly and the factorization of the diffusion operator are repeated on each of them, at 174s per call, against an IMEX run that amortizes all of it once over twenty steps. `pyMoDELib` removes the restart, and with it the dominant cost of a short splitting interval. Until then `FEM_EVERY` is as much a wall-clock knob as a numerical one.

2. **Guard the IMEX immobile solver against the next stiff term.** Section 8 shows the same failure was found twice, by symptom, and there is still no mechanism that would catch a third. Any new loss channel added to `solveImmobileClusters` must be recast as a first-order coefficient and placed in `lossN/lossC`; a cheap standing check is to run one case at `nSub` and at `10 nSub` and assert the answers agree, which turns a silent floor collapse into a visible regression.
3. **Revisit GPU offload only after the above.** With the core-evaluation count now the dominant cost and the LU already irrelevant at $n = 15$, the case for batched GPU integration rests entirely on how large q becomes in a live coupling — which is not yet known.

Files. `cpp_utils/dual.h`, `cpp_utils/rate_equations_core.h`, `cpp_utils/rate_equations.{h,cpp}`, `cpp_utils/jac_check.cpp`, `cpp_utils/parameters.h`, `code/solver.cpp`, `dislocluster_code/zerod/cpp_bridge.py`, `dislocluster_code/coupling/immobile.py`, `dislocluster_code/coupling/field.py` (new), `dislocluster_code/co` (new), `code/coupled_0d_3d_ZrMicro.ipynb`. The build now links `SUNDIALS::cvodes` rather than `SUNDIALS::cvsode`, because `CVodeQuadInit` and its companions exist only in `CVODES`; the header `<cvodes/cvodes.h>` was already being included.